

Graphs - Part 1

Introduction to Graphs

What is a Graph?

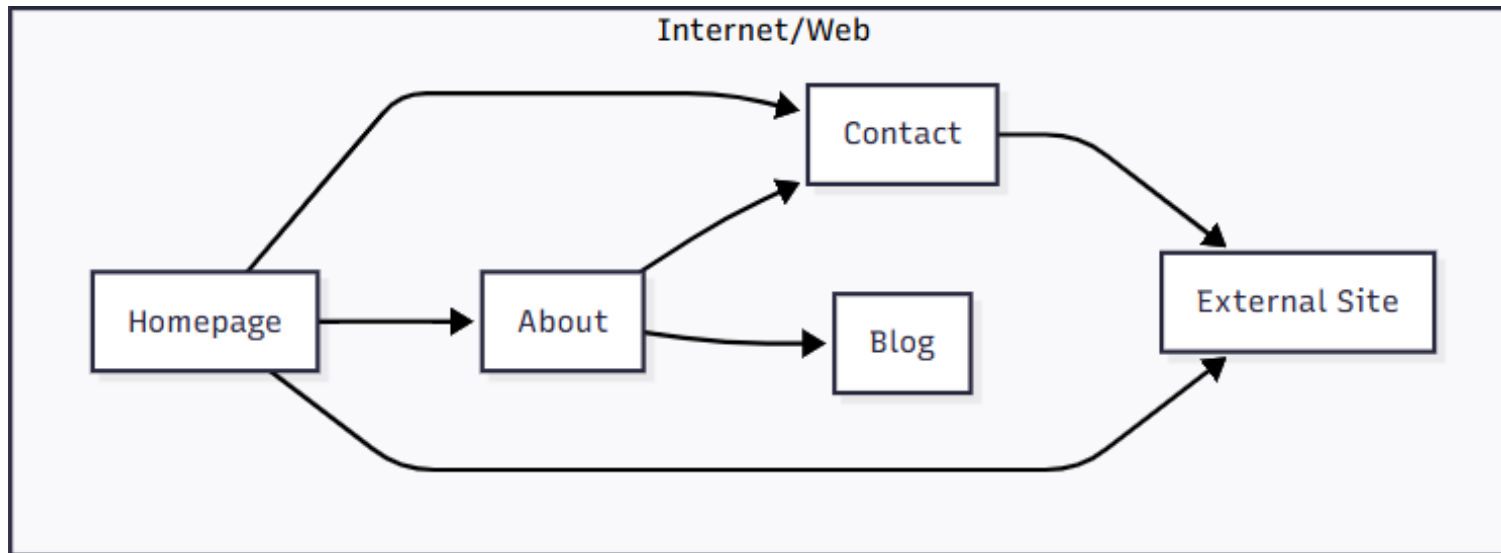
- A graph is a non-linear data structure consisting of **vertices** (nodes) and **edges** (connections between nodes).
- Unlike trees, graphs can have cycles and multiple paths between nodes.

Real-World Examples

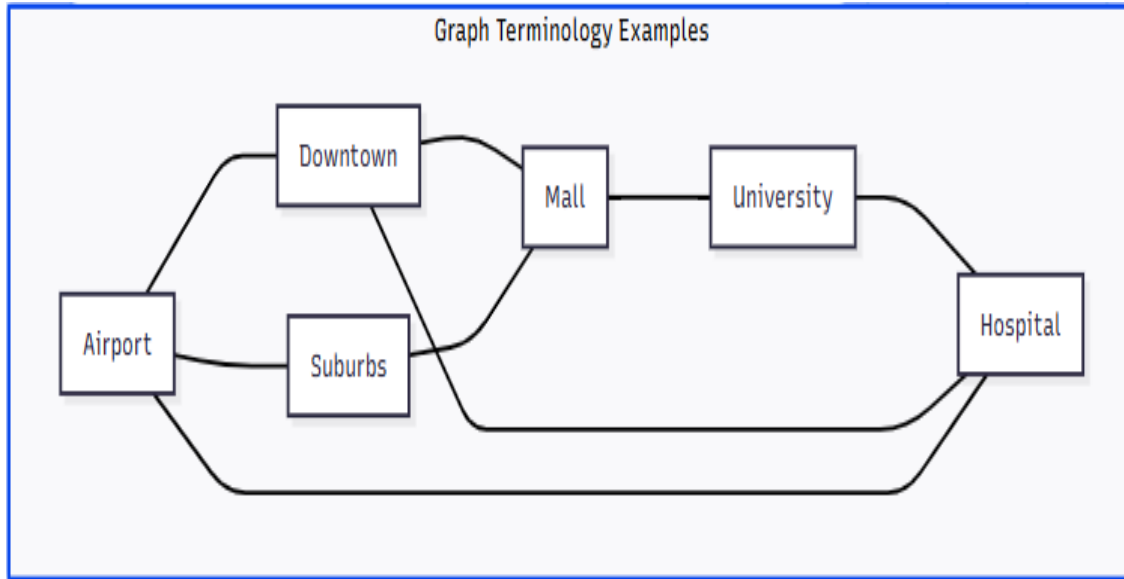
- Social Networks: Users are vertices, friendships are edges
- Maps: Cities are vertices, roads are edges
- Internet: Webpages are vertices, links are edges
- Computer Networks: Devices are vertices, connections are edges

Internet as a Graph

- Webpages are vertices, links are edges



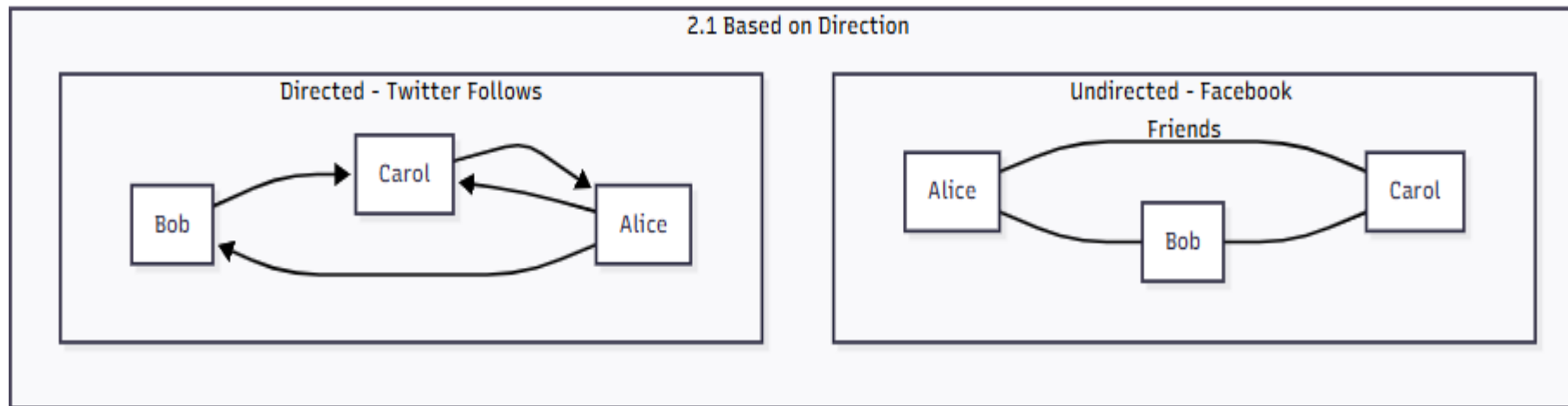
Basic Terminology



- **VERTEX/NODE:** Each location (Airport, Downtown, Mall, etc.)
- **EDGE:** Each road/route between locations“
- **ADJACENT:** Downtown & Mall are adjacent (connected by direct road)
- **DEGREE:** Downtown has degree 3 (connected to Airport, Mall, Hospital)
- **PATH:** Airport → Downtown → Mall → University“
- **CYCLE:** Airport → Downtown → Hospital → Airport"

Types of Graphs : Based on direction

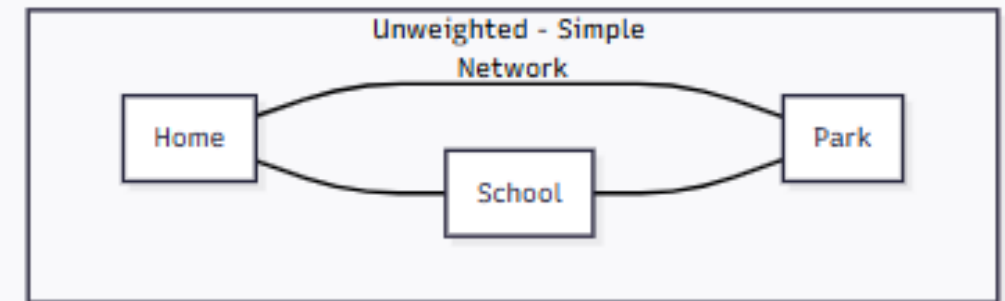
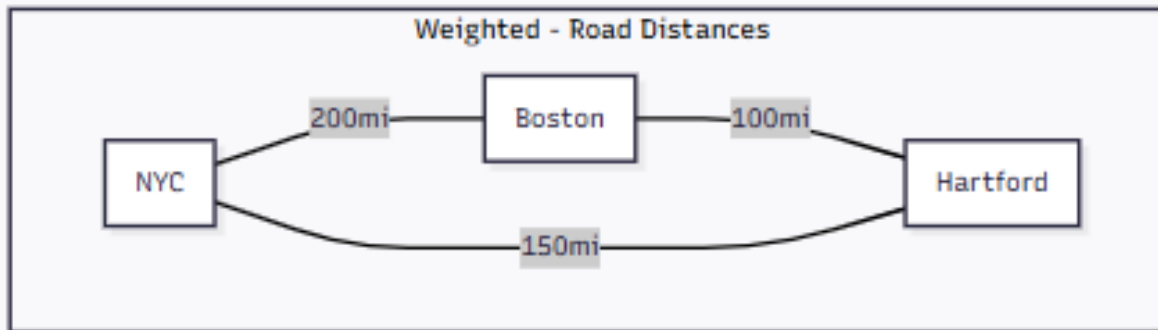
- **Undirected graph** : Edges have no direction (Facebook friendships)
 - If A connects to B, then B connects to A
- **Directed graph** : Edges have direction (Twitter followers)
 - If $A \rightarrow B$, it doesn't mean $B \rightarrow A$



Types of Graphs : Based on Weights

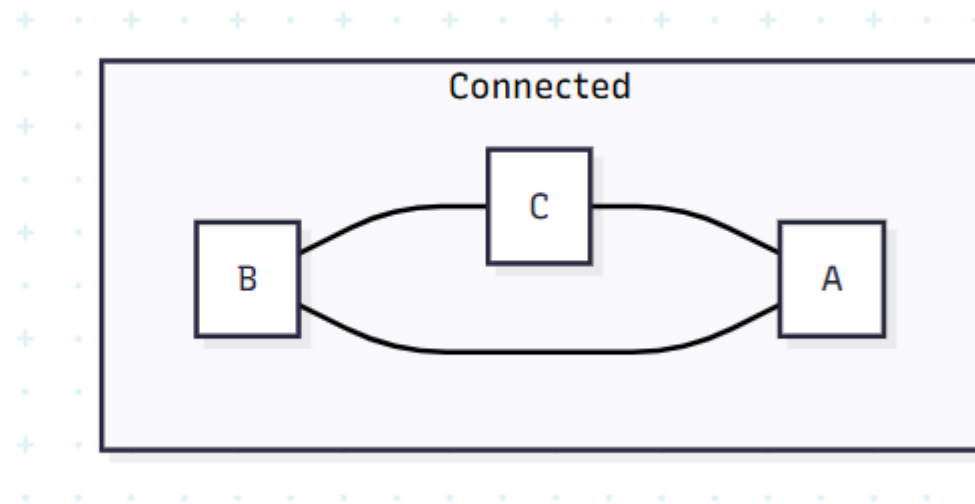
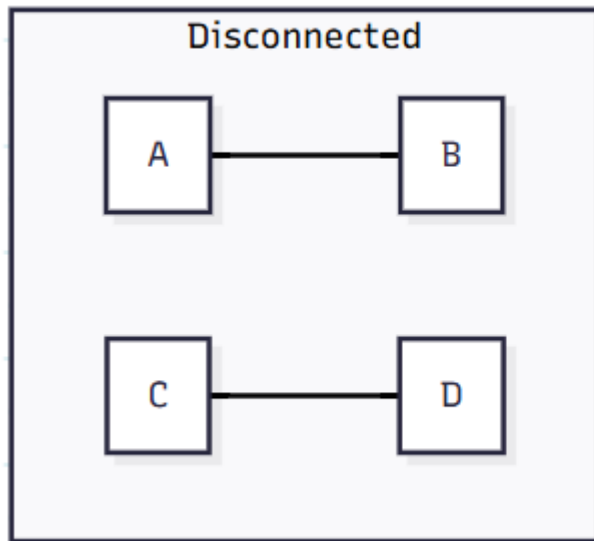
- **Unweighted Graph** : All edges have equal importance(Simple friendship network)
- **Weighted Graph** : Edges have associated weights/costs (Road network with distances)

2.2 Based on Weights



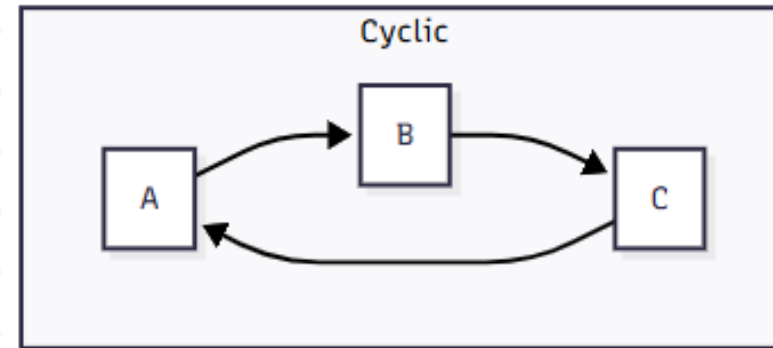
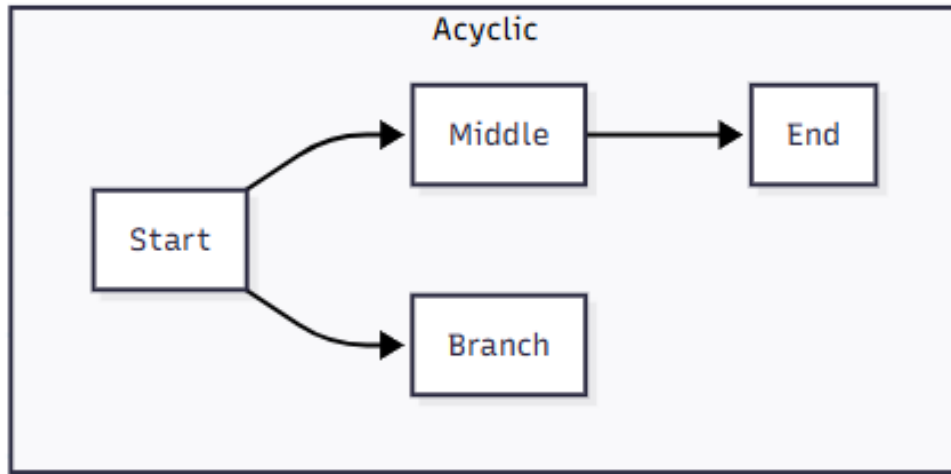
Types of Graphs : Special

- **Disconnected Graphs** : Some vertices are not reachable from the others
- **Connected Graphs** : Path exists between every pair of vertices



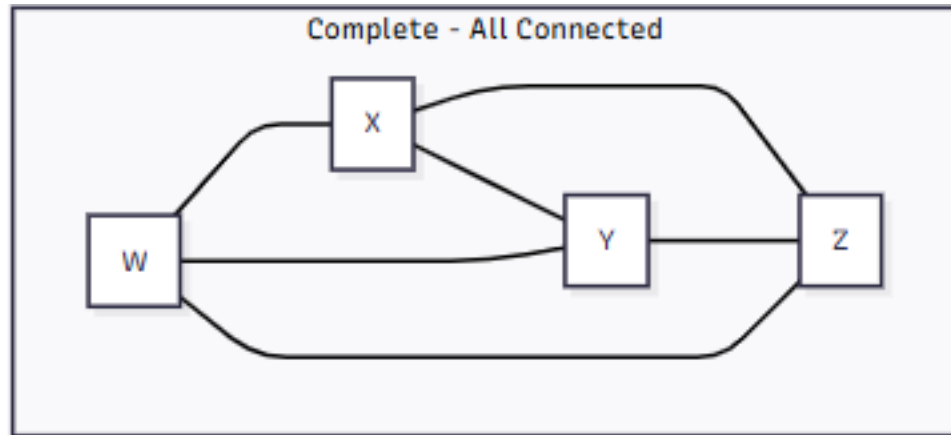
Types of Graphs : Special

- **Cyclic Graphs** : Contains at least one cycle
- **Acyclic Graphs** : Contains no cycles

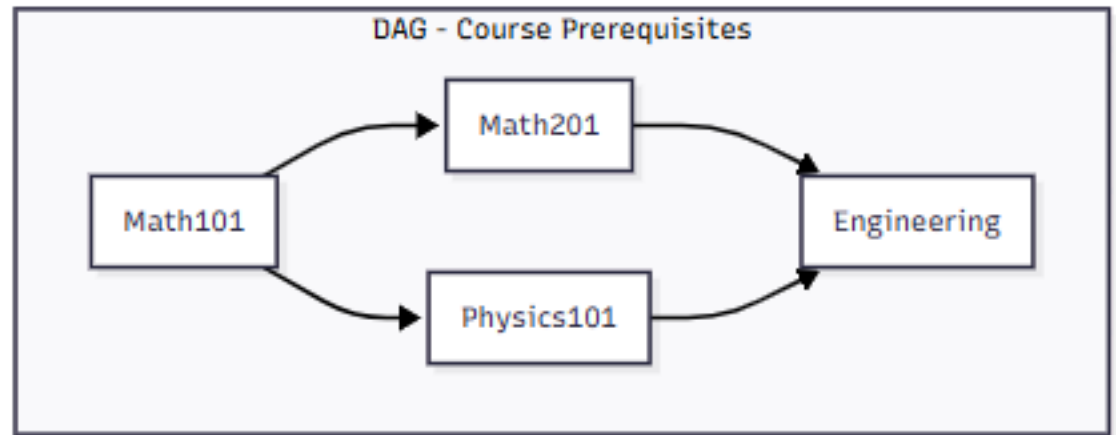


Types of Graphs : Special

- **Directed Acyclic Graphs (DAG)**
- **Complete Graph** : Every vertex is connected to every other vertex

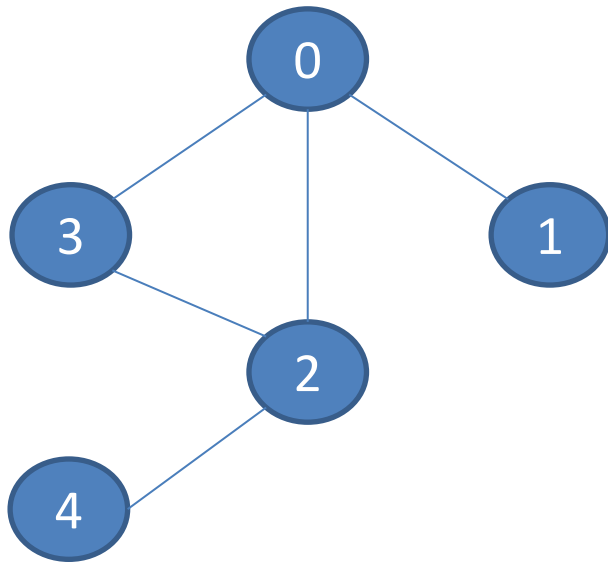


+
.
+
.
+
.



Graph Representations

- Adjacency List & Adjacency Matrix



Vertex	Connected to
0	[1 2 3]
1	[0]
2	[0 3 4]
3	[0 2]
4	[2]

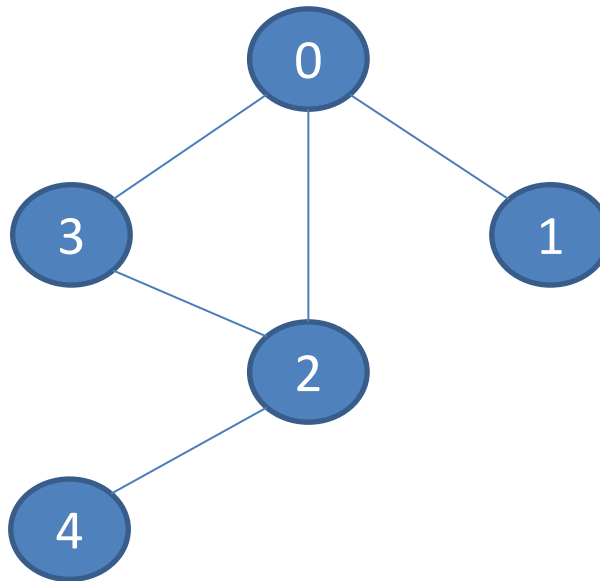
	0	1	2	3	4
0	0	1	1	1	0
1	1	0	0	0	0
2	1	0	0	1	1
3	1	0	1	0	0
4	0	0	1	0	0

Implementation :

Adjacency List vs. Matrix

- Java implementation/data structure:

Vertex	Connected to
0	[1 2 3]
1	[0]
2	[0 3 4]
3	[0 2]
4	[2]



	0	1	2	3	4
0	0	1	1	1	0
1	1	0	0	0	0
2	1	0	0	1	1
3	1	0	1	0	0
4	0	0	1	0	0

A list of list : `List<List<Integer>>`

A 2-D array : `matrix[i][j]`

Graphs - Part 2

Ajay Iyengar

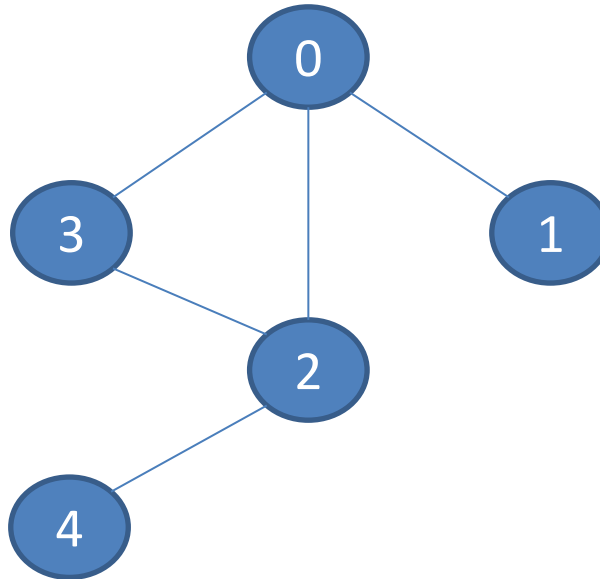
Email: iyengarajay@gmail.com

Implementation :

Adjacency List vs. Matrix

- Java implementation/data structure:

Vertex	Connected to
0	[1 2 3]
1	[0]
2	[0 3 4]
3	[0 2]
4	[2]



	0	1	2	3	4
0	0	1	1	1	0
1	1	0	0	0	0
2	1	0	0	1	1
3	1	0	1	0	0
4	0	0	1	0	0

A list of list : `List<List<Integer>>`

A 2-D array : `matrix[i][j]`

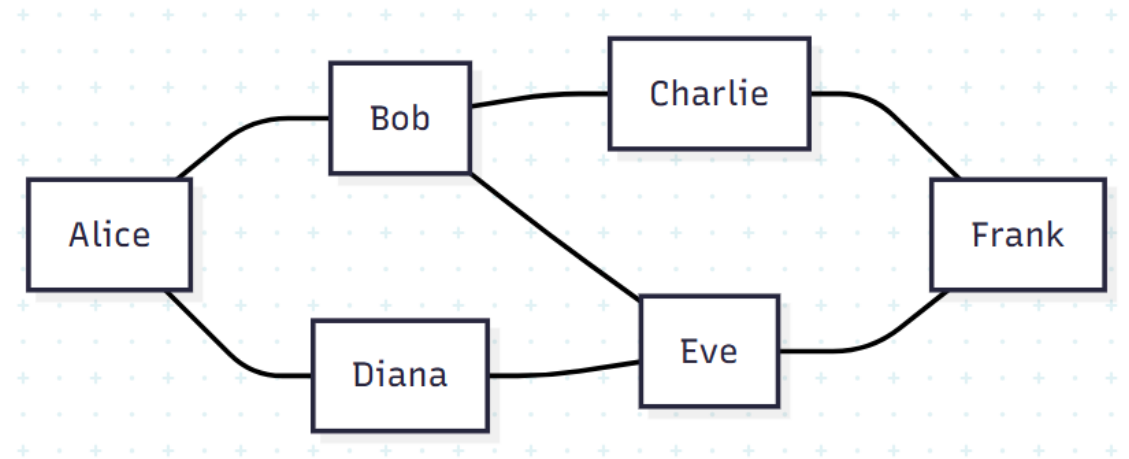
Graph Traversal Algorithms

- Depth-First Search
 - Explores as far as possible along each branch before backtracking
 - Goes deep into a path before exploring other possibilities
- Breadth-First Search
 - Explores all neighbor nodes at the current depth before moving to vertices at the next level.
 - Explores layer by layer

DFS Traversal Steps

```
1. Start: Alice (mark as visited)
2. Go to first neighbour: Bob (mark as visited)
3. From Bob, go deeper: Charlie (mark as visited)
4. From Charlie, go deeper: Frank (mark as visited)
5. From Frank, go deeper: Eve (mark as visited)
6. From Eve, go deeper: Diana (mark as visited)
7. Diana has no unvisited neighbors, backtrack to Eve
8. Eve has no more unvisited neighbors, backtrack to Frank
9. Frank has no more unvisited neighbors, backtrack to Charlie
10. Charlie has no more unvisited neighbors, backtrack to Bob
11. Bob has no more unvisited neighbors, backtrack to Alice
12. All nodes visited
```

DFS Order: Alice → Bob → Charlie → Frank → Eve → Diana

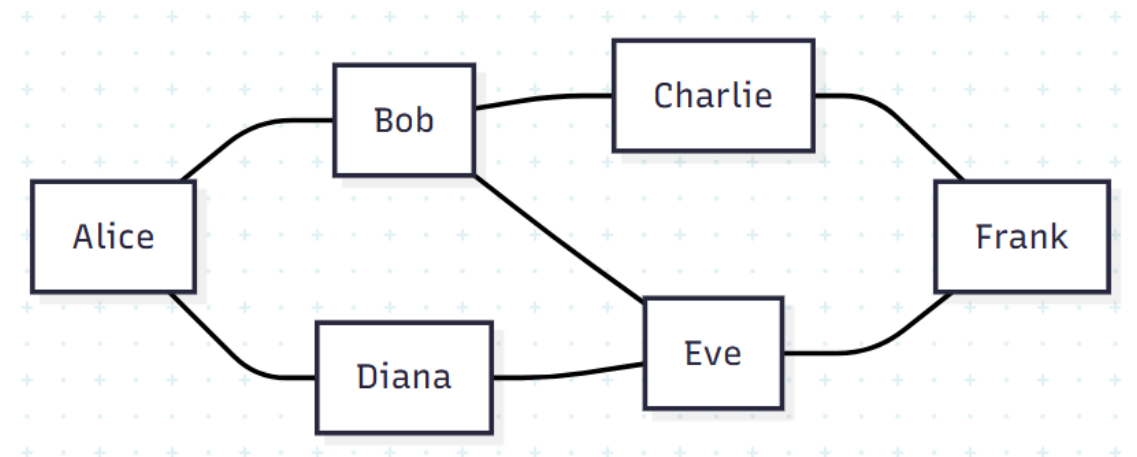


Algorithm: Go as deep as possible before backtracking

BFS Traversal Steps

```
1. Start: Alice (mark as visited)
2. Visit Alice's neighbors: Bob and Diana (mark as visited)
3. Visit Bob's unvisited neighbors: Charlie and Eve
4. Visit Diana's unvisited neighbors: (Eve already visited, so none)
5. Visit Charlie's unvisited neighbors: Frank
6. Visit Eve's unvisited neighbors: (all already visited)
7. Visit Frank's unvisited neighbors: (all already visited)

BFS Order: Alice → Bob → Diana → Charlie → Eve → Frank
```



Algorithm: Visit all neighbors at current level before going deeper

Technical Characteristics

DFS

- Uses Stack (or recursion)
- Time Complexity : $O(V+E)$
- Space Complexity : $O(V)$

BFS

- Uses Queue
- Time Complexity : $O(V+E)$
- Space Complexity: $O(V)$

Why is the time complexity $O(V+E)$?

- V = no of vertices and E = no of edges
- **The V Component (Vertices):**
 - We must **visit every vertex exactly once**
 - Each vertex gets marked as "visited"
 - This takes $O(V)$ time total
- **The E Component (Edges):**
 - We must **examine every edge at least once**
 - For each vertex, we look at all its edges to find neighbors
 - This takes $O(E)$ total time

Hence : $O(V+E)$

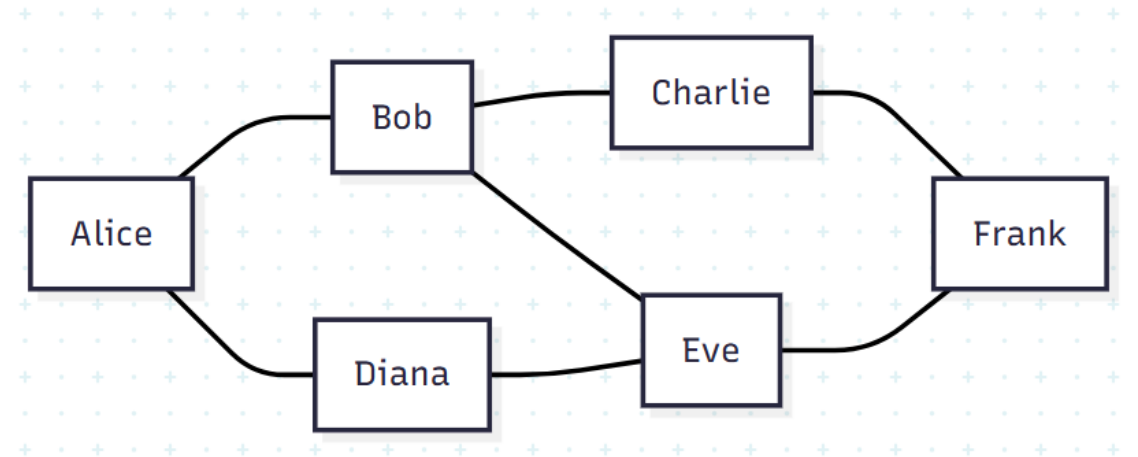
Example : $O(V+E)$

- V = no of vertices & E = no of edges, here:

$$V = 6 \text{ \& } E = 7$$

- During traversal:

- Visit 6 vertices: $O(6)$
- Examine 7 edges: $O(7)$
- Total: $O(6 + 7) = O(13) = O(V + E)$

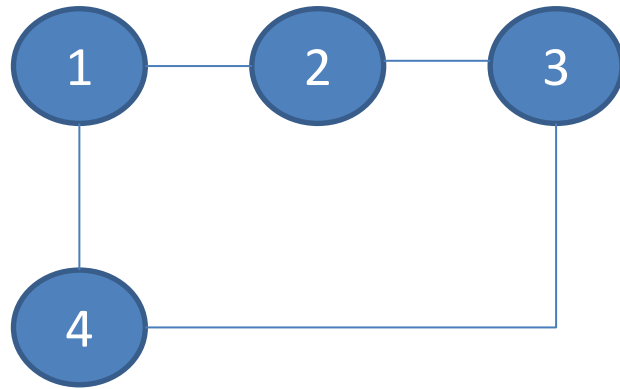


Hence : $O(V+E)$

Cycle detection in a graph

- Undirected graph
 - Use DFS, track parent and current, (if visited and current $\neq$ parent) = cycle
- Directed graph
 - Use DFS, maintain recursion stack to see if node visited again in current recursion stack

Cycle detection : Undirected graph



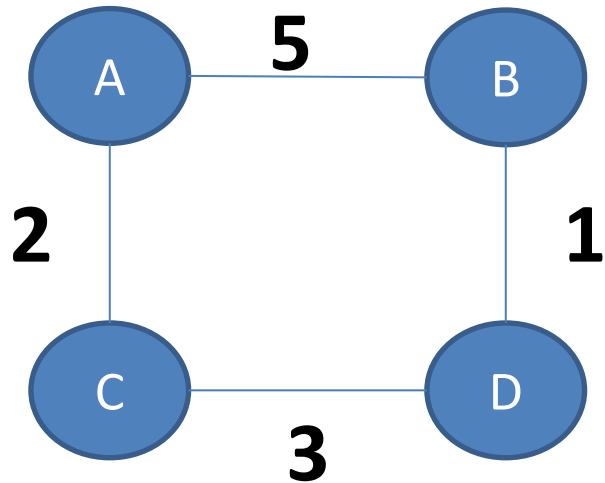
```
DFS(1) calls DFS(2)
├── DFS(2) calls DFS(3)
│   ├── DFS(3) calls DFS(4)
│   │   ├── DFS(4) finds neighbor 1 (visited, not parent)
│   │   │   ├── *** CYCLE DETECTED! ***
│   │   │   └── Returns TRUE
│   │   └── DFS(3) receives TRUE
│   │       ├── "My recursive call found a cycle!"
│   │       └── Returns TRUE
│   └── DFS(2) receives TRUE
│       ├── "My recursive call found a cycle!"
│       └── Returns TRUE
└── DFS(1) receives TRUE
    ├── "My recursive call found a cycle!"
    └── CYCLE DETECTED in the entire graph!
```

Minimum Spanning Tree

- A spanning tree of a graph is a subgraph that:
 - Connects all vertices in the original graph
 - Contains no cycles(it's a tree)
 - Uses exactly $(V-1)$ edges
- A **minimum** spanning tree is the spanning tree with the smallest total weight of all edges

MST: Example

- 4 cities: A,B,C,D
- Edges represent roads
- Numbers represent the cost(in millions) to build the roads
- **Connect *all cities* with minimum total cost.**
- Some possible spanning trees
 - Tree 1: A-C (2) + C-D (3) + D-B (1) = Total cost: 6
 - Tree 2: A-B (5) + A-C (2) + C-D (3) = Total cost: 10
 - Tree 3: A-C (2) + A-B (5) + B-D (1) = Total cost: 8
 - Few more
- ***The MST is Tree 1 with total cost 6.***



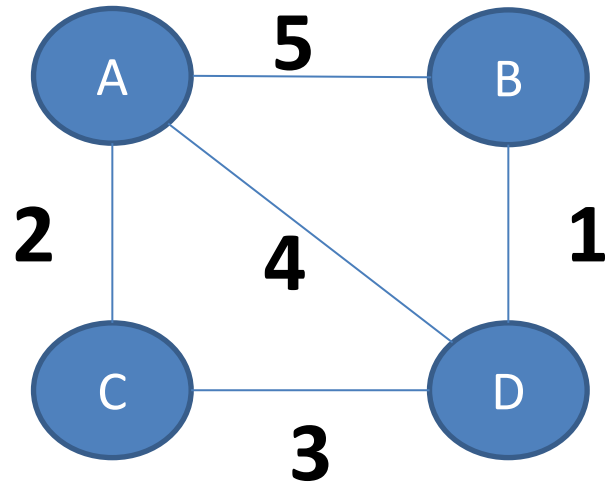
We don't need to build ALL possible roads to connect all cities. We just need enough roads so that you can travel from any city to any other city

Algorithms to find MST

- **Kruskal's Algorithm:** “Start with cheapest edges globally”
 - Sort all edges by weight
 - Keep adding the cheapest edge that doesn't create a cycle
- **Prim's Algorithm:** “Grow the tree from one vertex”
 - Start from any vertex
 - Keep adding the cheapest edge that connects to a new vertex

Kruskal's algorithm

- Pick cheapest edges first, avoid cycles



- 5 possible roads: A-B: cost 5, A-C: cost 2, A-D: cost 4, B-D: cost 1, C-D: cost 3



Step 1: List ALL edges sorted by weight (cheapest first)

B-D: 1 , A-C: 2 , C-D: 3 , A-D: 4 , A-B: 5



Step 2: Start with no edges. Add edges one by one IF they don't create a cycle.



Add B-D (1): No cycle yet ✓

```
  B
  |
  1
  |
  D
```



Add A-C (2): No cycle yet ✓

```
A    B
|    |
2    1
|    |
C    D
```



Add C-D (3): No cycle yet ✓

```
A    B
|    |
2    1
|    |
C--3--D
```

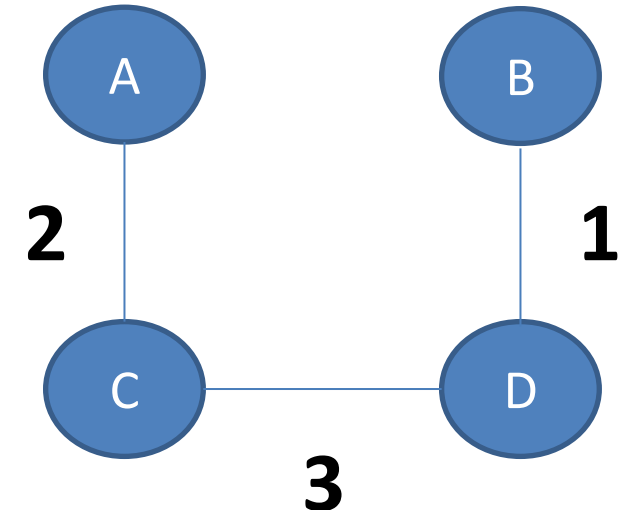


Skip A-D (4): Would create cycle A-C-D-A ✗



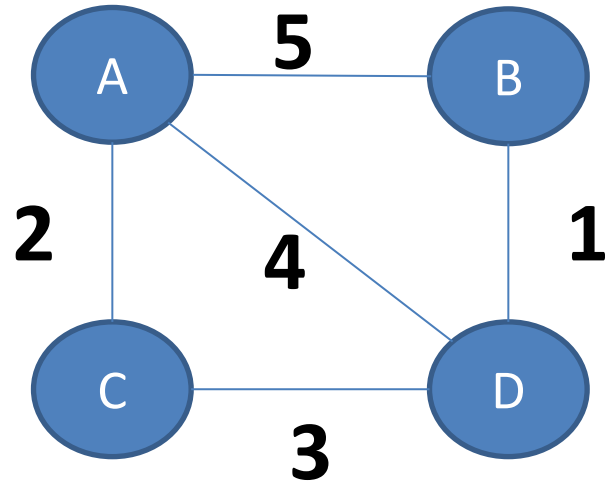
Skip A-B (5): Would create cycle A-C-D-B-A ✗

Result: MST with edges B-D, A-C, C-D (total: 6)



Prim's algorithm

- Pick cheapest edges first, avoid cycles



- 5 possible roads: A-B: cost 5, A-C: cost 2, A-D: cost 4, B-D: cost 1, C-D: cost 3



Step 1: Start from any vertex (let's pick A)
Current tree: {A}
Available edges from A: A-B(5), A-C(2), A-D(4)



Step 2: Add the cheapest edge that connects to a NEW vertex
Choose A-C (2) - cheapest from A
Current tree: {A, C}

```
A
|
2
|
C
```



Step 3: Look at ALL edges from current tree vertices {A, C}
Available edges: A-B(5), A-D(4), C-D(3)
Choose C-D (3) - cheapest edge to new vertex D
Current tree: {A, C, D}

```
A
|
2
|
C--3--D
```



Step 4: Look at ALL edges from current tree vertices {A, C, D}
Available edges: A-B(5), D-B(1)
Choose D-B (1) - cheapest edge to new vertex B

Final tree: {A, B, C, D}

```
A      B
|      |
2      1
|      |
C--3--D
```



Key Differences Summary

Kruskal's

- Looks at ALL edges globally
- Picks cheapest edges that don't create cycles
- Can work on disconnected components simultaneously
- Needs cycle detection (Union-Find data structure)

Prim's

- Grows tree from one starting point
- Always picks cheapest edge from current tree to outside
- Tree stays connected throughout
- Simpler to implement with priority queue

Graphs – Part 3

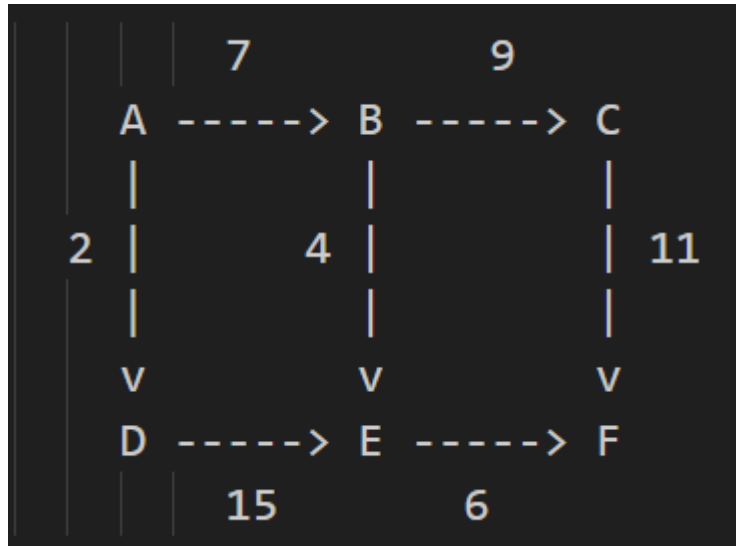
Ajay Iyengar

Email: iyengarajay@gmail.com

Dijkstra's Algorithm

- What does it do ?
 - Find the shortest path from **ONE source** vertex to **all other vertices** in a weighted graph.
- Key Insight
 - Always expand to the nearest unvisited vertex first (greedy approach)
- The problem it solves
 - **Given** : A weighted graph (directed or undirected) & a starting vertex
 - **Find** : Shortest distance from source to every other vertex & the actual path taken to reach each vertex

Dijkstra's Algorithm Walkthrough



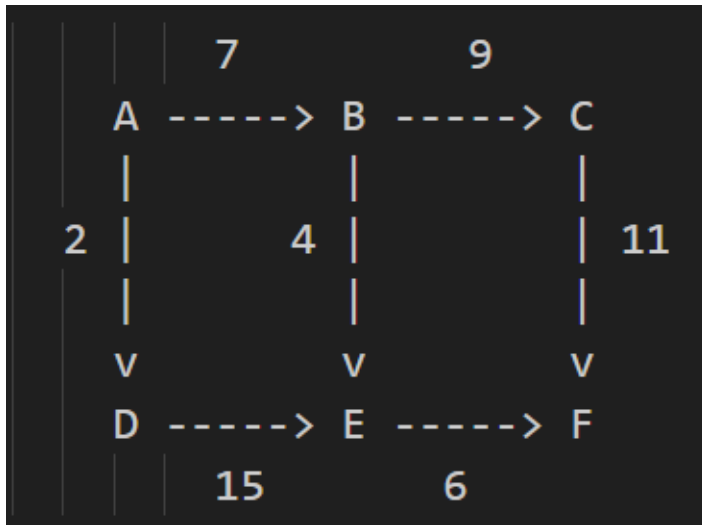
```
A → [(B, 7), (D, 2)]
B → [(C, 9), (E, 4)]
C → [(F, 11)]
D → [(E, 15)]
E → [(F, 6)]
F → []
```

Node	Distance	Visited	Previous
A	0	×	-
B	∞	×	-
C	∞	×	-
D	∞	×	-
E	∞	×	-
F	∞	×	-

Source Node : A

Priority Queue: [(A, 0)]

Dijkstra's Algorithm Walkthrough : Step 1



Node	Distance	Visited	Previous
A	0	✗	-
B	∞	✗	-
C	∞	✗	-
D	∞	✗	-
E	∞	✗	-
F	∞	✗	-

Step 1 : Process Node A

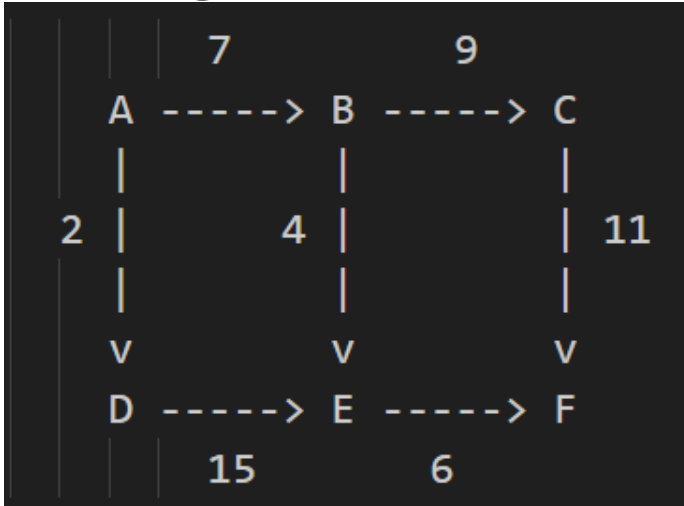
Neighbors to Update:

- B: $0 + 7 = **7**$ (better than ∞)
- D: $0 + 2 = **2**$ (better than ∞)

Priority Queue: [(D, 2), (B, 7)]

Node	Distance	Visited	Previous
A	0	✓	-
B	7	✗	A
C	∞	✗	-
D	2	✗	A
E	∞	✗	-
F	∞	✗	-

Dijkstra's Algorithm Walkthrough : Step 2



Node	Distance	Visited	Previous
A	0	✓	-
B	7	✗	A
C	∞	✗	-
D	2	✗	A
E	∞	✗	-
F	∞	✗	-

Step 2 : Process Node D(distance = 2)

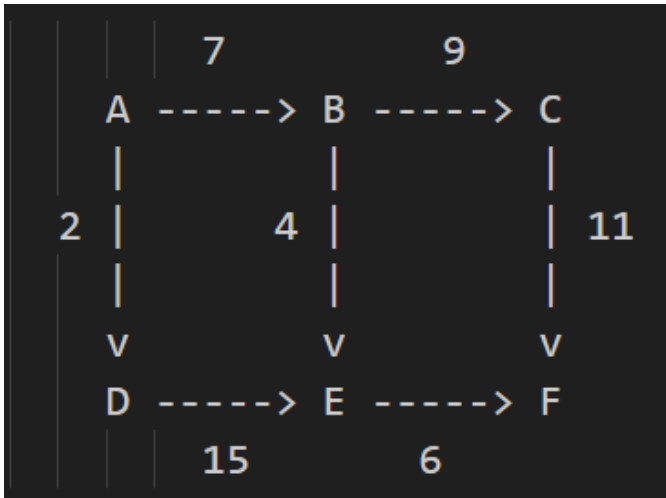
Neighbors to Update:

E: $2 + 15 = 17$ (better than ∞)

Priority Queue: [(B, 7), (E, 17)]

Node	Distance	Visited	Previous
A	0	✓	-
B	7	✗	A
C	∞	✗	-
D	2	✓	A
E	17	✗	D
F	∞	✗	-

Dijkstra's Algorithm Walkthrough : Step 3



Node	Distance	Visited	Previous
A	0	✓	-
B	7	✗	A
C	∞	✗	-
D	2	✓	A
E	17	✗	D
F	∞	✗	-

Step 3 : Process Node B(distance = 7)

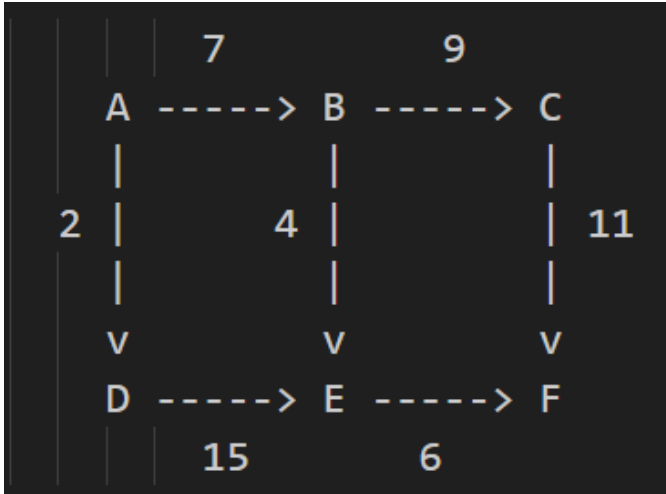
Neighbors to Update:

- **C**: $7 + 9 = \text{**16**}$ (better than ∞)
- **E**: $7 + 4 = \text{**11**}$ (better than 17)

Priority Queue: [(E, 11), (C, 16), (E, 17)]

Node	Distance	Visited	Previous
A	0	✓	-
B	7	✓	A
C	16	✗	B
D	2	✓	A
E	11	✗	B
F	∞	✗	-

Dijkstra's Algorithm Walkthrough : Step 4



Node	Distance	Visited	Previous
A	0	✓	-
B	7	✓	A
C	16	✗	B
D	2	✓	A
E	11	✗	B
F	∞	✗	-

Step 3 : Process Node E(distance = 11)

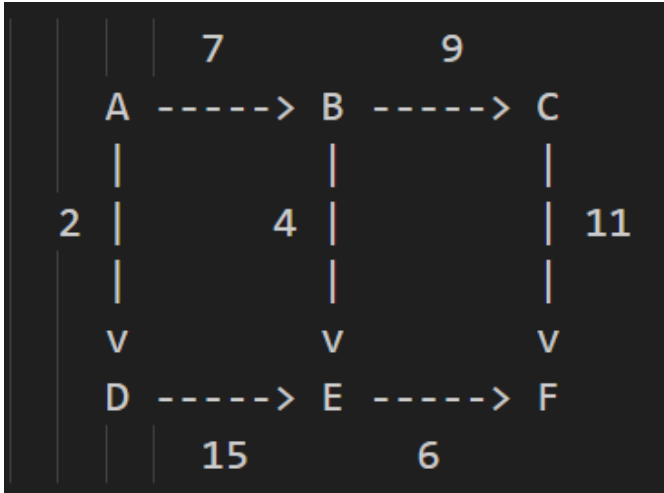
Neighbors to Update:

F: $11 + 6 = 17$ (better than ∞)

Priority Queue: [(C, 16), (F, 17), (E, 17)]

Node	Distance	Visited	Previous
A	0	✓	-
B	7	✓	A
C	16	✗	B
D	2	✓	A
E	11	✓	B
F	17	✗	E

Dijkstra's Algorithm Walkthrough : Step 5



Node	Distance	Visited	Previous
A	0	✓	-
B	7	✓	A
C	16	✗	B
D	2	✓	A
E	11	✓	B
F	17	✗	E

Step 3 : Process Node C (distance = 16)

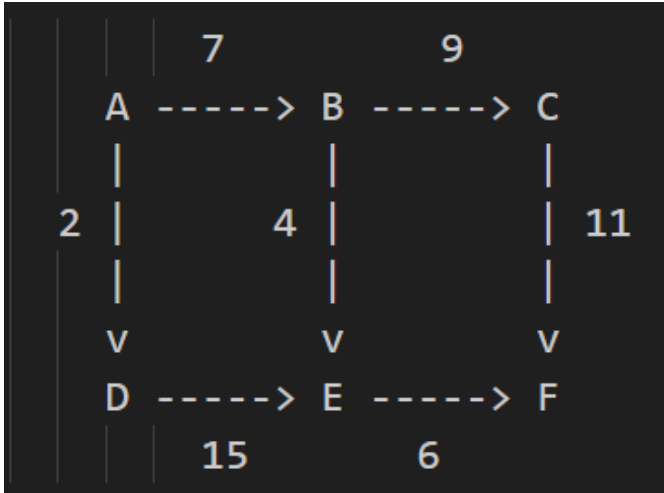
Neighbors to Update:

F: $16 + 11 = \textbf{27}$ (worse than 17), *No update*

Priority Queue: [(F, 17), (E, 17)]

Node	Distance	Visited	Previous
A	0	✓	-
B	7	✓	A
C	16	✓	B
D	2	✓	A
E	11	✓	B
F	17	✗	E

Dijkstra's Algorithm Walkthrough : Step 6



Node	Distance	Visited	Previous
A	0	✓	-
B	7	✓	A
C	16	✓	B
D	2	✓	A
E	11	✓	B
F	17	✗	E

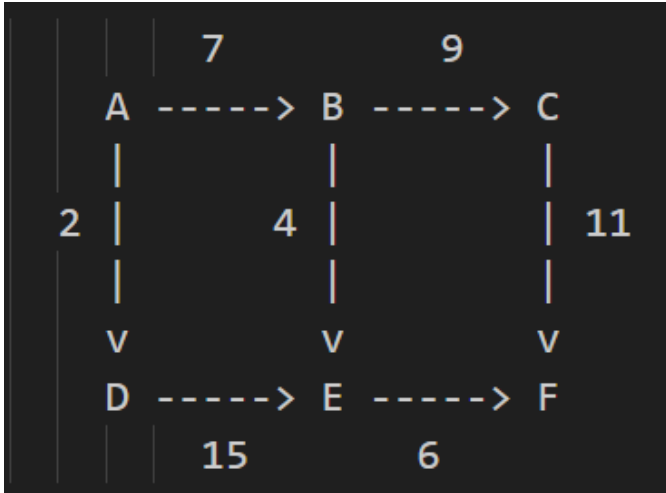
Step 6 : Process Node F (distance = 17)

Neighbors to Update: None

Priority Queue: [(E, 17)]

Node	Distance	Visited	Previous
A	0	✓	-
B	7	✓	A
C	16	✓	B
D	2	✓	A
E	11	✓	B
F	17	✓	E

Dijkstra's Algorithm Walkthrough : Step 7

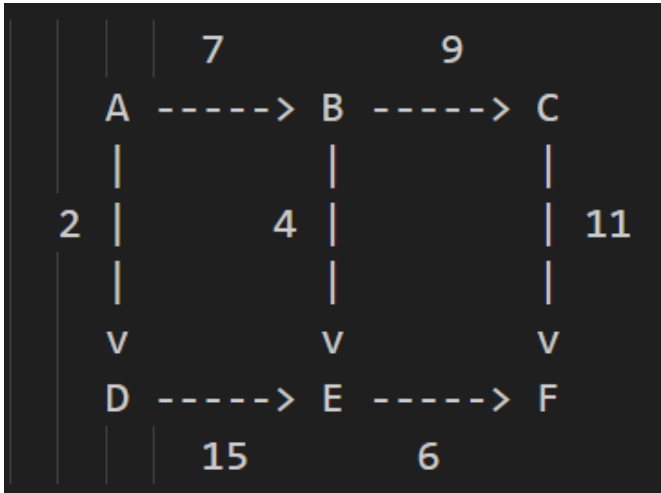


Node	Distance	Visited	Previous
A	0	✓	-
B	7	✓	A
C	16	✓	B
D	2	✓	A
E	11	✓	B
F	17	✓	E

Step 7: Process Node E but this is already visited & now the priority queue is empty.

Priority Queue: []

Dijkstra's Algorithm : Final Result



Node	Distance	Visited	Previous
A	0	✓	-
B	7	✓	A
C	16	✓	B
D	2	✓	A
E	11	✓	B
F	17	✓	E

Shortest Distances from A:

- **A** → **A**: 0
- **A** → **B**: 7 (Path: A → B)
- **A** → **C**: 16 (Path: A → B → C)
- **A** → **D**: 2 (Path: A → D)
- **A** → **E**: 11 (Path: A → B → E)
- **A** → **F**: 17 (Path: A → B → E → F)

Dijkstra's Algorithm : Key Takeaways

- **Greedy Approach:** Always process the node with minimum distance
- **Relaxation:** Update distances if a shorter path is found
- **Priority Queue:** Efficiently fetches the next minimum distance node
- **Time Complexity:** $O((V + E) \log V)$ with min-heap

Dijkstra's : Real-World Applications

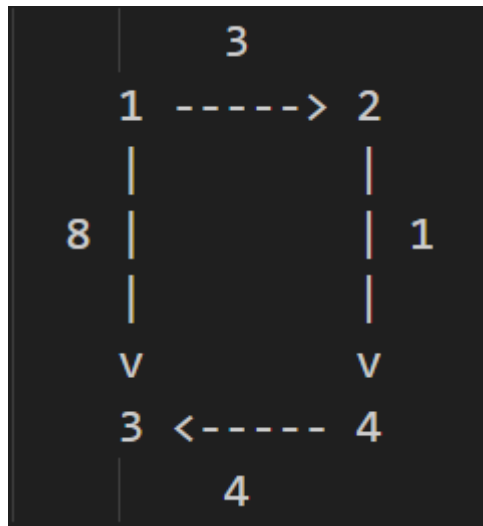
- **GPS Navigation** - From your location, what's the shortest route to every landmark?
- **Delivery Services** - From warehouse, find optimal routes to all customers
- **Network Routing** - From this server, find fastest path to all other servers
- **Finance** - From starting position, find lowest-cost path to all outcomes

Floyd–Warshall Algorithm

- What does it do ?
 - Finds the ***shortest paths between ALL pairs of vertices*** in a weighted graph
- Algorithm in One Formula :
For each vertex k (as intermediate):
For each pair (i, j) :
$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

Can I make the path from i to j shorter by going through k ?

Floyd–Warshall Algorithm - Example

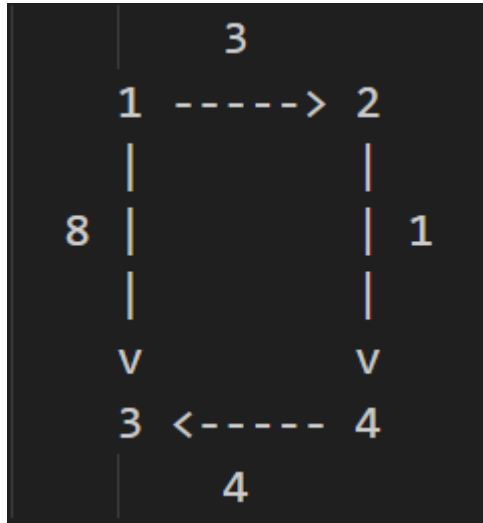


	1	2	3	4
1	0	3	8	∞
2	∞	0	∞	1
3	∞	∞	0	∞
4	∞	∞	4	0

- $\text{dist}[i][j]$ = edge weight if direct edge exists
- $\text{dist}[i][i]$ = 0 (distance to itself)
- $\text{dist}[i][j]$ = ∞ if no direct edge

Floyd–Warshall Algorithm – Step 1

Use vertex 1 as intermediate and check if we can improve any path:

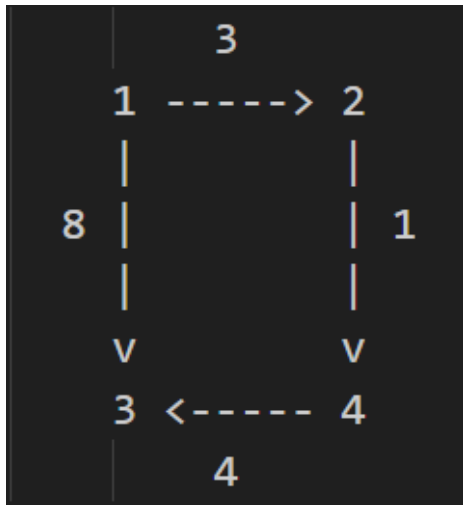


	1	2	3	4
1	0	3	8	∞
2	∞	0	∞	1
3	∞	∞	0	∞
4	∞	∞	4	0

No improvements found and hence for step 2, distance matrix will be the same.

Floyd–Warshall Algorithm – Step 2

Before : Use vertex 2 as intermediate and check if we can improve any path :



	1	2	3	4
1	0	3	8	∞
2	∞	0	∞	1
3	∞	∞	0	∞
4	∞	∞	4	0

Path 1→4:

Current: $\text{dist}[1][4] = \infty$

Via 2: $\text{dist}[1][2] + \text{dist}[2][4] = 3 + 1 = 4$

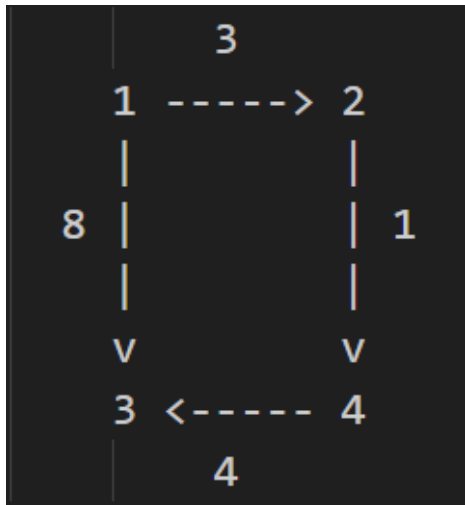
Result: $4 < \infty$, hence update to 4!

Path discovered: 1→2→4 with distance 4

	1	2	3	4
1	0	3	8	**4** ★
2	∞	0	∞	1
3	∞	∞	0	∞
4	∞	∞	4	0

Floyd–Warshall Algorithm – Step 3

Before : Use vertex 3 as intermediate and check if we can improve any path :

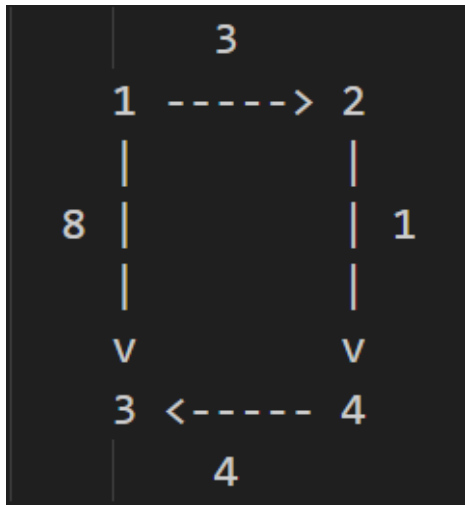


	1	2	3	4
1	0	3	8	4
2	∞	0	∞	1
3	∞	∞	0	∞
4	∞	∞	4	0

No improvements found and hence for step 3, distance matrix will be the same.

Floyd-Warshall Algorithm – Step 4

Before : Use vertex 4 as intermediate and check if we can improve any path :



	1	2	3	4
1	0	3	8	4
2	∞	0	∞	1
3	∞	∞	0	∞
4	∞	∞	4	0

Path 2→3:

Current: $\text{dist}[2][3] = \infty$

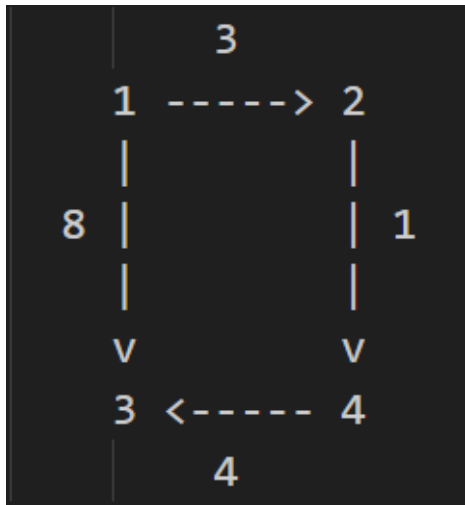
Via 4: $\text{dist}[2][4] + \text{dist}[4][3] = 1 + 4 = 5$

Result: $5 < \infty$, Update to 5!

Path discovered: 2→4→3 with distance 5

	1	2	3	4
1	0	3	8	4
2	∞	0	5 ★	1
3	∞	∞	0	∞
4	∞	∞	4	0

Floyd–Warshall Algorithm – Final



	1	2	3	4
1	0	3	8	4
2	∞	0	5	1
3	∞	∞	0	∞
4	∞	∞	4	0

From vertex 1: Can reach 2(3), 3(8), 4(4)

From vertex 2: Can reach 3(5), 4(1)

From vertex 3: Can only reach itself

From vertex 4: Can reach 3(4)

Dijkstra's vs. Floyd-Warshall

Aspect	Dijkstra's	Floyd-Warshall
Solves	Single source to all	All pairs
Approach	Greedy	Dynamic Programming
Negative edges	Cannot handle	Can handle
Time Complexity	$O((V+E) \log V)$	$O(V^3)$
Best for	One source, sparse graph	All pairs, dense graph
Use when	From city A, find shortest paths to everywhere	Between ANY two cities, what's the shortest path?

Introduction to Trees

Ajay Iyengar

Email: iyengarajay@gmail.com

Introduction to Trees

- Why Trees ? The limitation of Linear Data Structures
- Scenario 1 : Searching in a database
 - You have 1 million records in an array
 - Linear search: $O(n)$ - potentially 1 million comparisons
 - Even with binary search: $O(\log n)$ - but insertion/deletion is $O(n)$
- Scenario 2 : Organizing a File System
 - How to represent folders and subfolders in an array or linked list ?
 - How do you efficiently represent hierarchy ?
 - How do you navigate parent-child relationships ?

Why Trees ?

- Scenario 3 : Auto-complete in Search Engines
 - As you type "prog", it suggests "program", "programming", "progress"
 - Linear structures would require scanning all words
 - Need something that can branch based on prefixes

The Problem & the Solution

Linear data structures (arrays, linked lists, stacks, queues) are excellent for sequential access but struggle with:

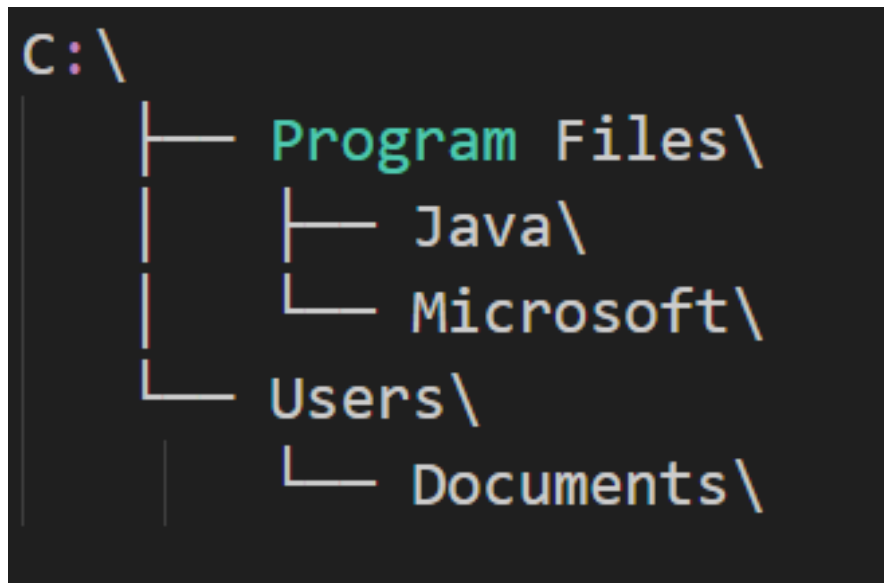
- Hierarchical relationships
- Efficient searching with dynamic data
- Representing multi-way branching
- Balancing speed of insertion and search

The Solution: Trees - a non-linear, hierarchical data structure.

Trees : Real World Analogies

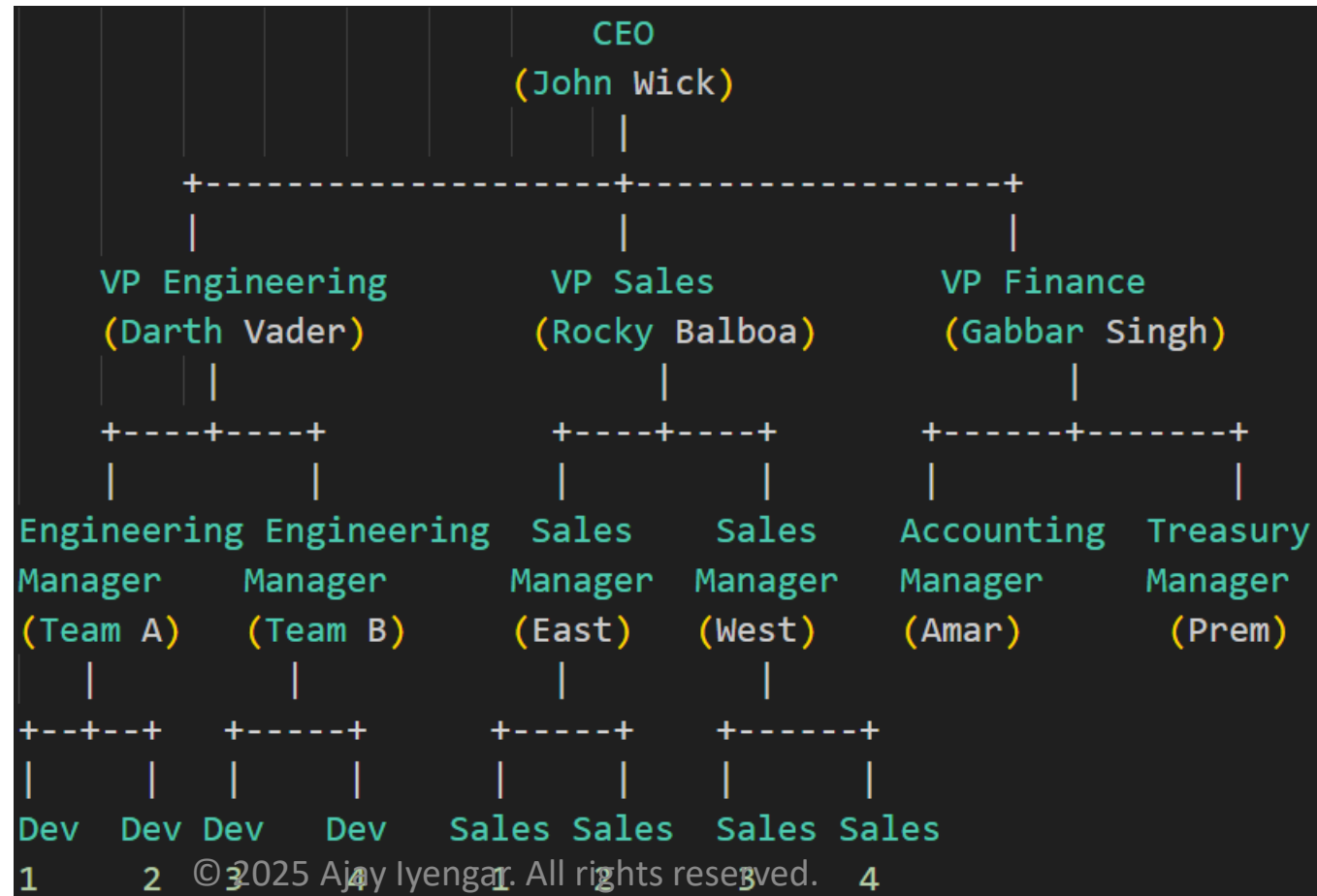
Trees are everywhere in computing and real life –

1. File System : Computer's folder structure



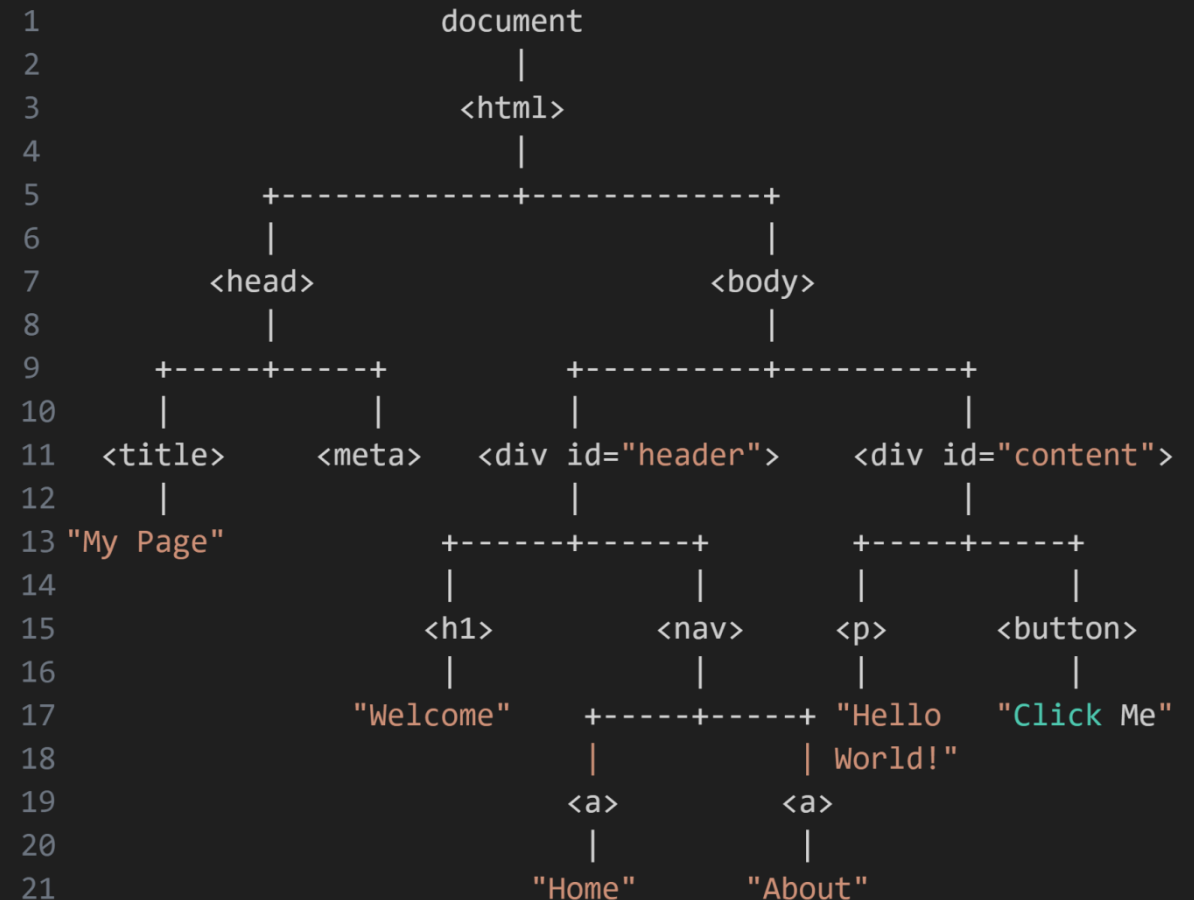
Trees : Real World Analogies

Organizational Hierarchy : Company Structures(CEO - VP's – Managers - Developers)



Trees : Real World Analogies

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <div id="header">
      <h1>Welcome</h1>
      <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
      </nav>
    </div>
    <div id="content">
      <p>Hello World!</p>
      <button>Click Me</button>
    </div>
  </body>
</html>
```



Dom as a Tree

Trees : Real World Analogies

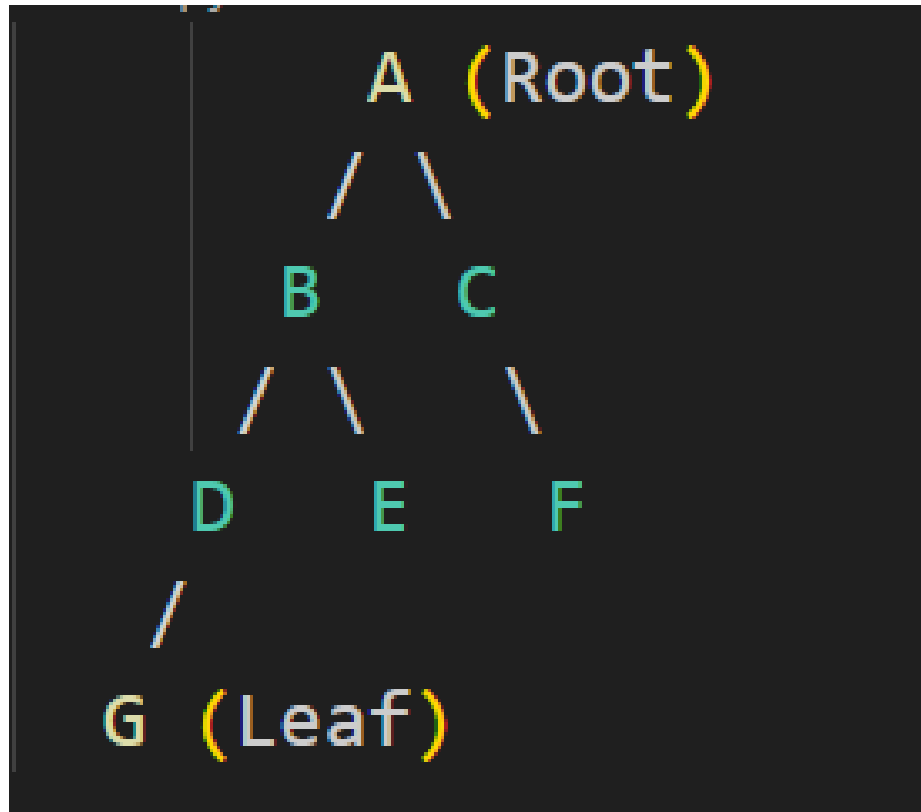
Decision Tree – AI/ML, Game Theory

```

1  CURRENT STATE
2
3      +---+---+---+
4      | X | X |   |
5      +---+---+---+
6      | O | O |   |
7      +---+---+---+
8      |   |   |   |
9      +---+---+---+
10
11      |
12      +-----+-----+
13
14      |
15      +---+---+---+
16      | X | X | X | ✓ WIN!
17      +---+---+---+
18      | O | O |   |
19      +---+---+---+
20      |   |   |   |
21      +---+---+---+
22
23      |
24      GAME OVER
25      X WINS!
26      Score: +10
27      ✓ BEST CHOICE
28
29      |
30      +---+---+---+
31      | X | X |   |
32      +---+---+---+
33      | O | O |   |
34      +---+---+---+
35      |   |   |   |
36      +---+---+---+
37
38      |
39      O's Next Move
40      O plays [1,2]
41      |
42      +---+---+---+
43      | X | X | O |
44      +---+---+---+
45      | O | O | X | ✓ Blocked
46      +---+---+---+
47      |   |   |   |
48      +---+---+---+
49
50      |
51      Game continues...
52      X may lose or draw
53      Score: 0 or -10
54      X BAD CHOICE

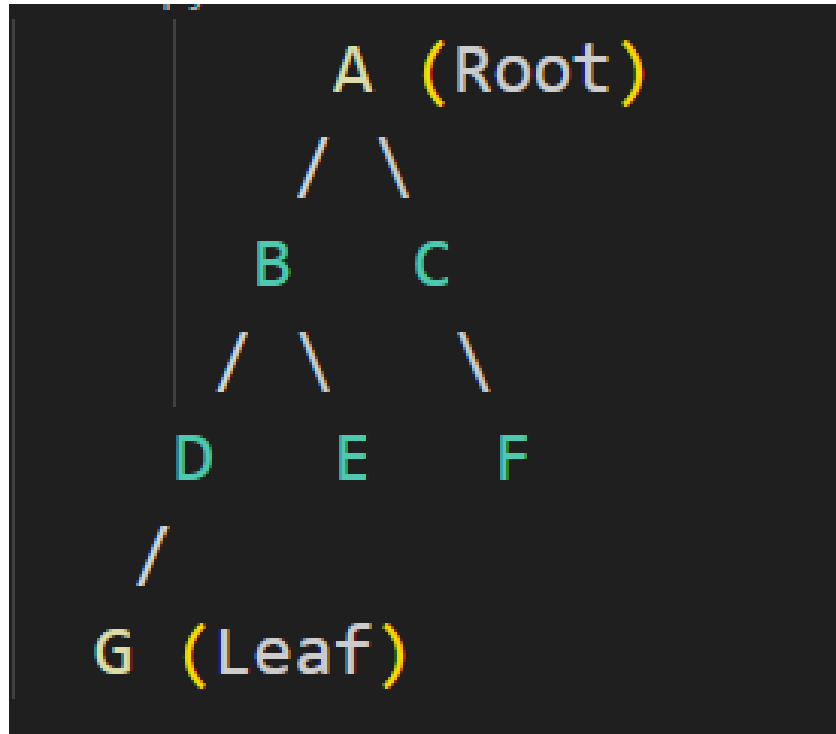
```

Basic Tree Terminology



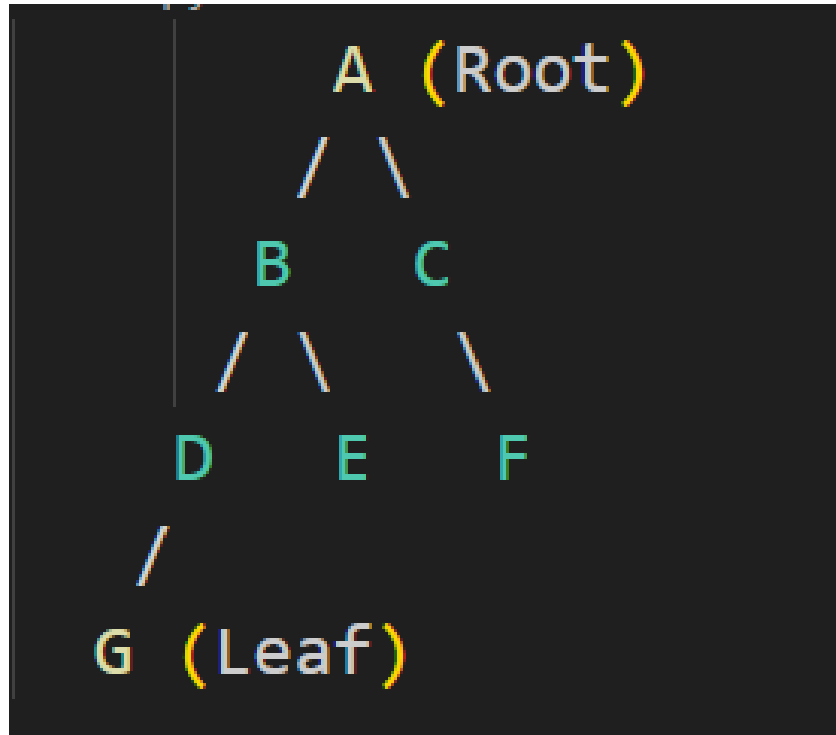
- **Node**: Each element in the tree (A, B, C, D, E, F, G)
- **Root**: The topmost node with no parent (A)
- **Parent**: Node that has children (A is parent of B and C)
- **Child**: Node that has a parent (B and C are children of A)

Basic Tree Terminology



- **Siblings:** Nodes with the same parent (B and C are siblings)
- **Leaf/External Node:** Node with no children (E, F, G)
- **Edge:** Connection between two nodes (there are 6 edges above)

Basic Tree : Measurement terms

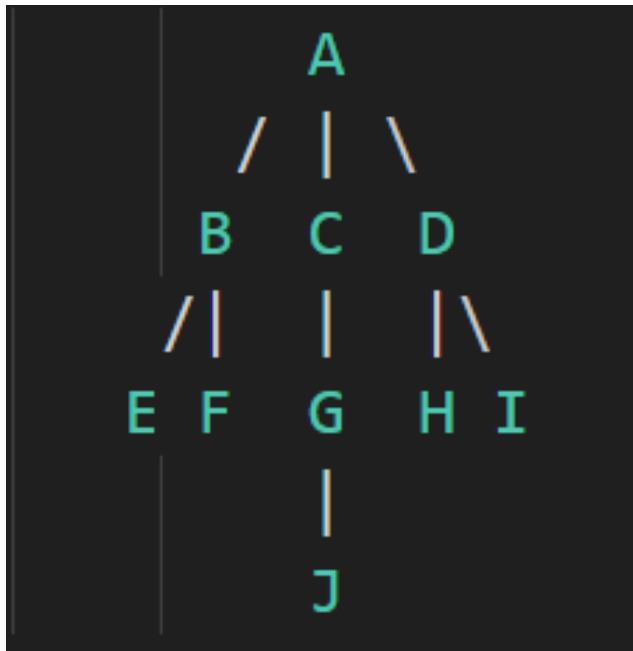


- **Depth of a Node:** Number of edges from root to that node
 - Depth of A = 0
 - Depth of B = 1
 - Depth of G = 3
- **Height of a Node:** Number of edges on longest path from that node to a leaf
 - Height of G = 0 (leaf)
 - Height of D = 1
 - Height of B = 2
 - Height of A = 3

Height of Tree: Height of the root node
(3 in above example)

Types of Trees : General Tree(N-ary Tree)

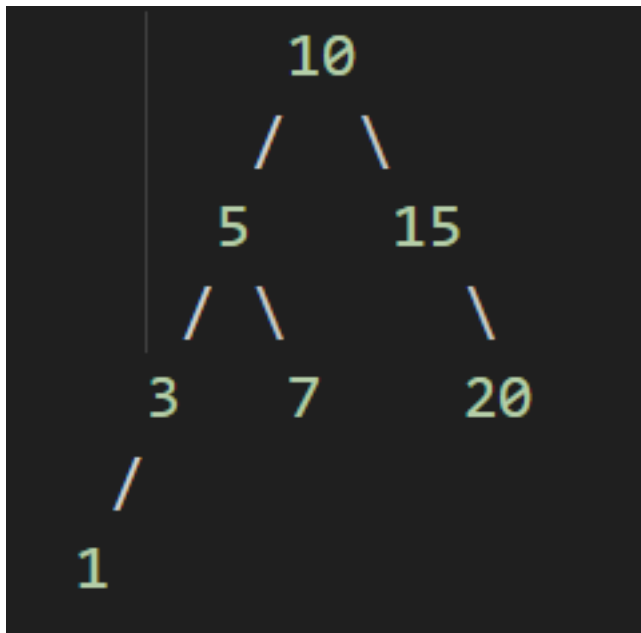
- A tree where each node can have any number of children (0 to N).



- **Properties:**
 - No restriction on number of children
 - Most flexible tree structure
 - Used in file systems, organizational charts
- **Use Case:** File system directories where a folder can contain any number of subfolders and files.

Binary Trees

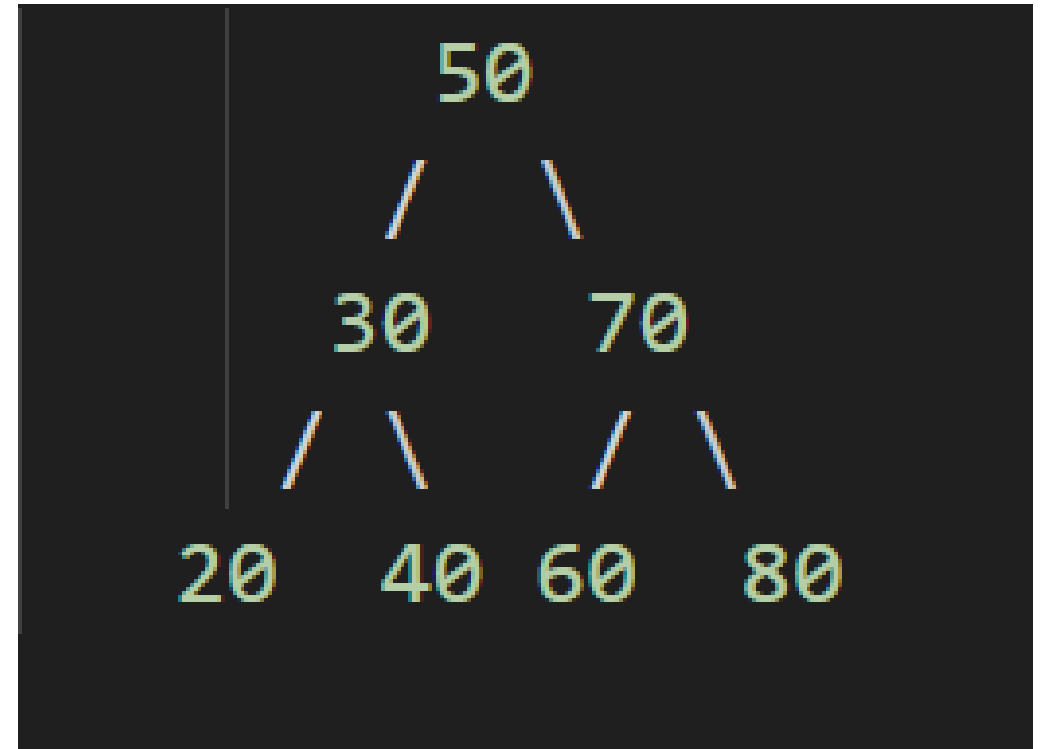
- A tree where each node has at most 2 children (left child and right child).



- **Properties:**
 - Each node has 0, 1, or 2 children
 - Children are labeled as left and right
 - Order matters: left $\neq$ right
- **Use Case:** Expression trees, Huffman coding trees, basic tree operations learning.

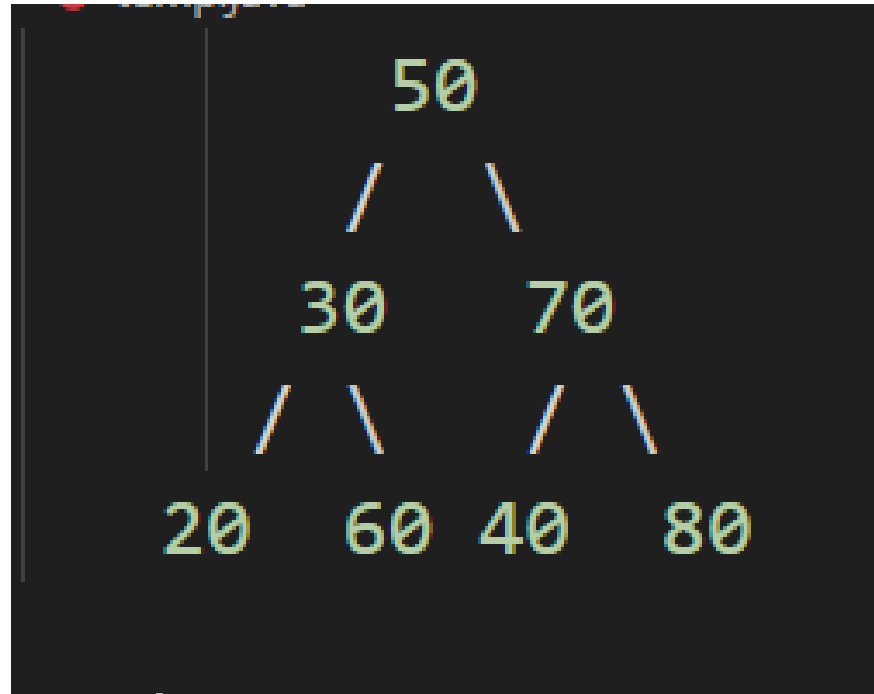
Binary Search Trees*

- A binary tree where for each node:
 - All values in the left subtree are less than the node's value
 - All values in the right subtree are greater than the node's value
 - Both left and right subtrees are also BSTs



Use cases: Databases indexing, searching with dynamic insertions/deletions, symbol tables in compilers.

Not A Binary Search Tree (BST)

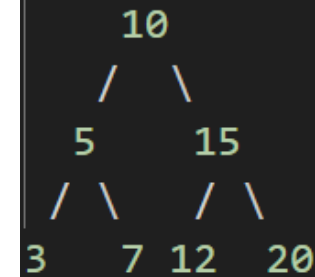


60 in left subtree but $60 > 50$ - violates BST property &
40 in right subtree – violates BST property

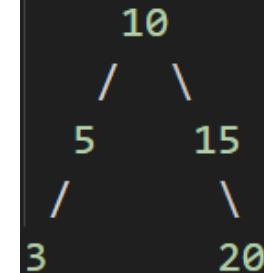
Full Binary Tree (Strict Binary Tree)

- A binary tree where every node has either 0 or 2 children. No node has exactly 1 child
- **Properties**
 - Every internal node has exactly 2 children
 - All leaves can be at different levels
 - If there are L leaves, there are $(L-1)$ internal nodes

Valid Full Binary Tree:



Not a Full Binary Tree:

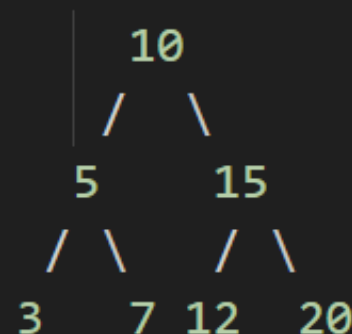


(Nodes 5 and 15 have only 1 child each - violates full binary tree property)

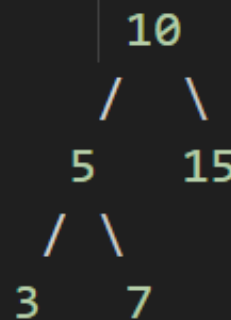
Perfect Binary Tree

- A binary tree where all internal nodes have exactly 2 children **AND** all leaf nodes are at the same level.
- **Properties**
 - Most restrictive binary tree type
 - All Perfect trees are also Full and Complete
 - Perfectly balanced

Perfect Binary Tree:



NOT Perfect (but Full):



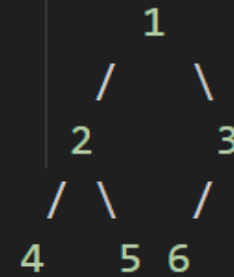
(This is Full but not Perfect - leaves at different levels)

Complete Binary Tree

- A binary tree where **all levels are completely filled except possibly the last level**, and the last level is filled from left to right.

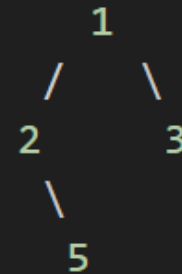
Some books & articles may refer to the Same as Almost Complete Binary Trees

Complete Binary Tree:



Last level filled from left to right

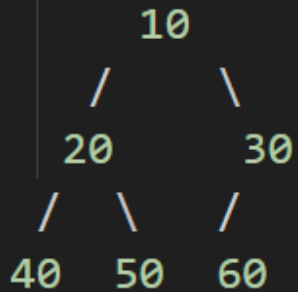
Not a Complete Binary Tree:



Left child missing, right child exists

Array representation : Complete Binary Trees

Complete Binary Tree:



Stored in Array (level order):

Index: 0 1 2 3 4 5
Array: [10, 20, 30, 40, 50, 60]

1. Root first
2. Then its children
3. Then grandchildren
4. Exactly how arrays store data

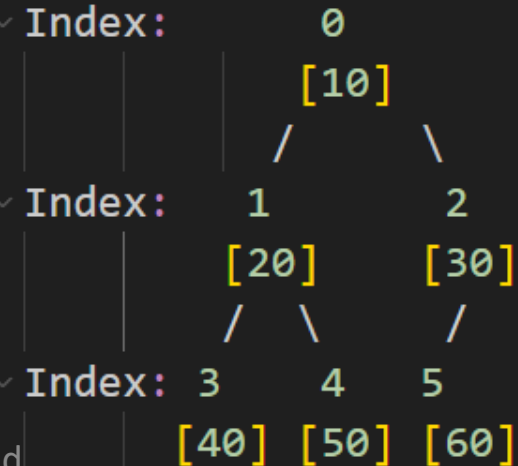
Index relationship:

Left child = $2*i + 1$

Right child = $2*i + 2$

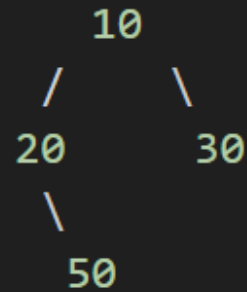
Parent = $(i - 1) / 2$

Index: 0 1 2 3 4 5
Array: [10, 20, 30, 40, 50, 60]



Array representation : Works only for Complete Binary Trees

Not Complete:



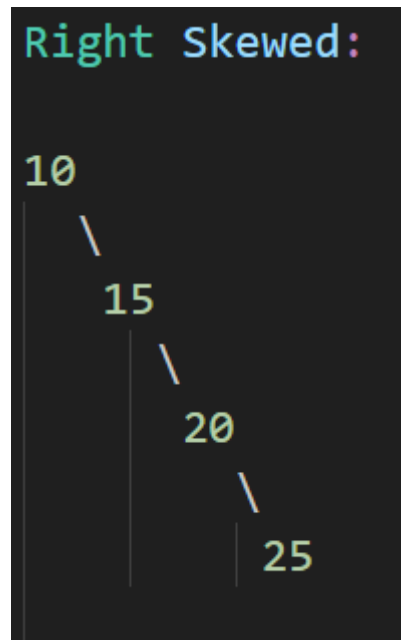
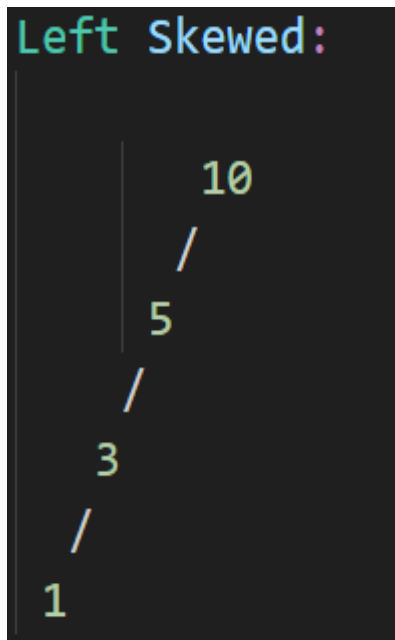
Attempted Array:

Index:	0	1	2	3	4
Array:	[10,	20,	30,	?,	50]

1. Gap at index 3
2. Index formulas break
3. Wasted space

Skewed Tree

- A tree where each parent node has only one child. Essentially becomes a linked list.



- **Properties:**
 - Worst case scenario for search operations
 - $O(n)$ time complexity for all operations
 - Height = Number of nodes - 1

This is what happens to an unbalanced BST when you insert sorted data. Understanding this motivates the need for self-balancing trees.

Balanced Trees

- Balanced trees maintain height $\approx \log(n)$ to ensure $O(\log n)$ operations even in worst case.



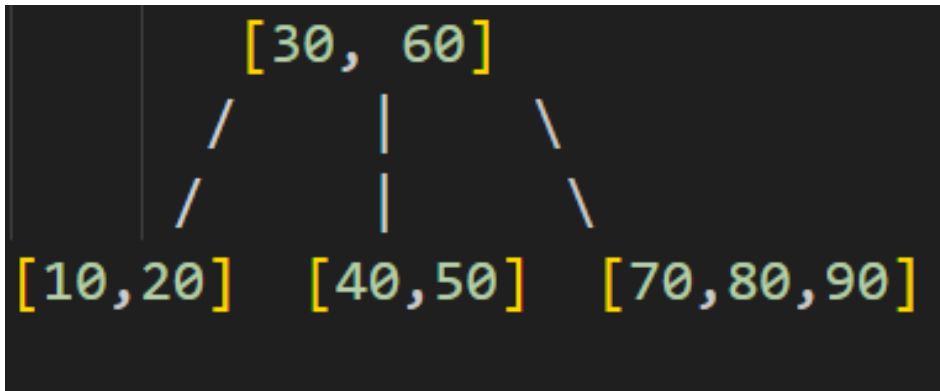
- AVL Tree:**
 - Self-balancing BST where the height difference between left and right subtrees (balance factor) is at most 1 for every node
 - Each node's $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$

Balanced Trees

- **Red-Black Tree:**
 - Self-balancing BST with color properties (red/black nodes) ensuring tree remains approximately balanced
- **Use Cases:**
 - Java's TreeMap, TreeSet, Linux kernel scheduling, C++ STL map.
- **Properties**
 - Every node is either red or black
 - Root is always black
 - Red nodes cannot have red children
 - Every path from root to leaf has same number of black nodes

B-Tree

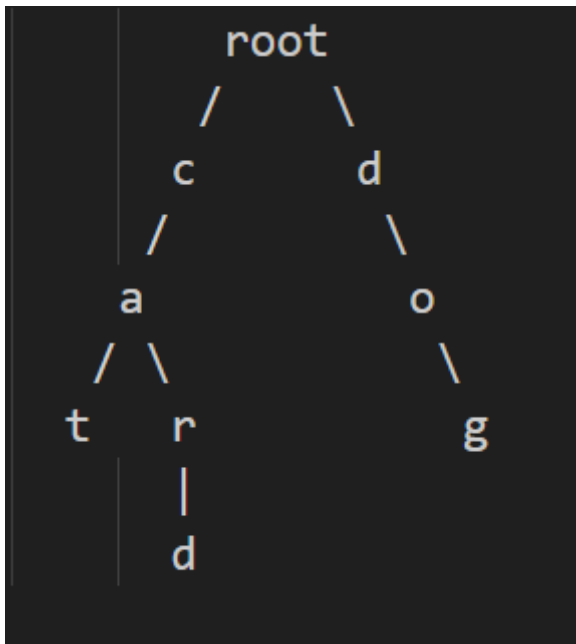
- Self-balancing search tree where nodes can have multiple keys and more than 2 children. Designed for disk storage systems



- **Properties**
 - Each node can contain multiple keys (not just one)
 - All leaves at the same level
 - Optimized for systems that read/write large blocks of data
 - Minimizes disk I/O operations
- **Use cases:** Database indexing (MySQL, PostgreSQL use B+ Trees), file systems (NTFS, ext4)

Trie

- Tree structure where each node represents a character, used for efficient string storage and retrieval
- Storing "cat", "car", "card", "dog":



- **Properties**
 - Each path from root to node represents a prefix
 - Common prefixes share paths
 - Fast string lookup: $O(L)$ where L is string length
- **Use cases :** Auto-complete, spell checkers, dictionary implementations.

BST Operations

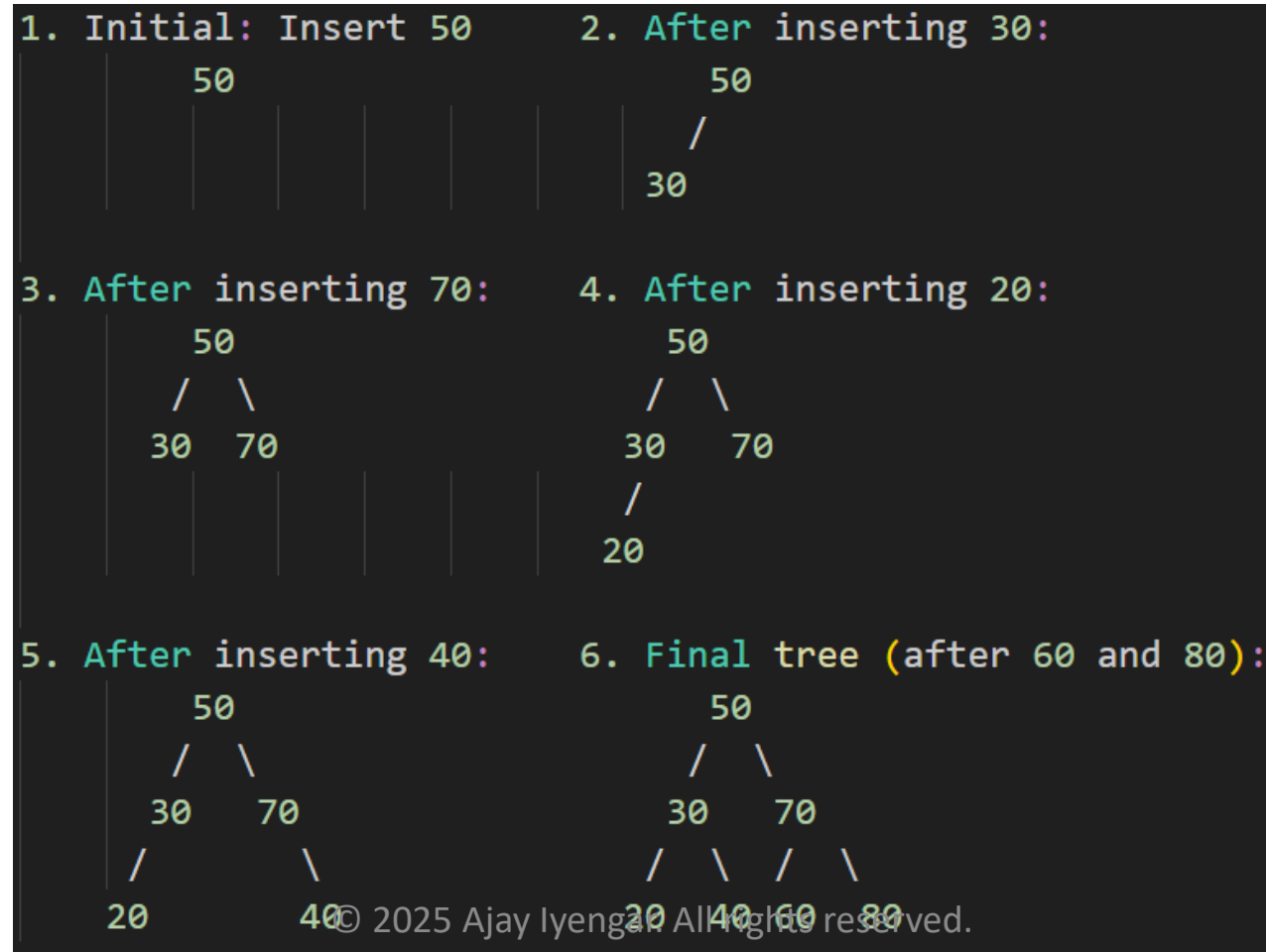
- A **Binary Search Tree (BST)** is a special type of binary tree where each node follows a simple rule:

Left child < Parent node < Right child

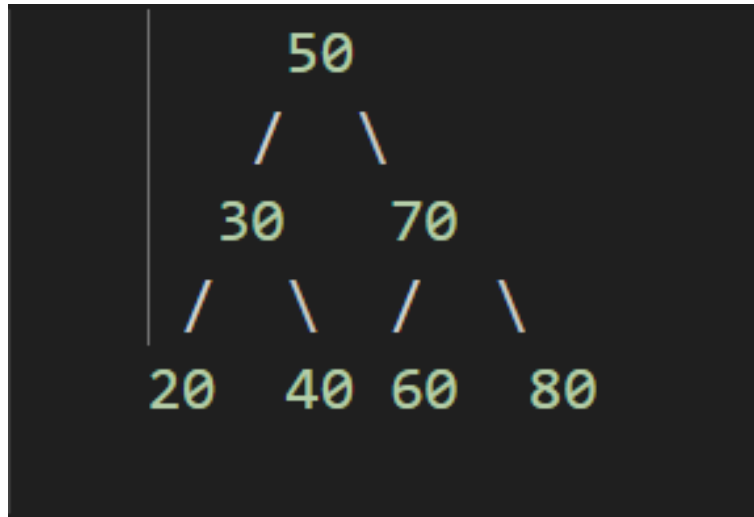
- This property must be true for **every node** in the tree
- This makes BSTs incredibly efficient for searching, insertion, and deletion operations.
- Unlike arrays or linked lists where searching takes $O(n)$ time, a well-balanced BST allows you to search for elements in $O(\log n)$ time - similar to binary search!

Creating a BST

Let's say we want to insert these numbers in order: 50, 30, 70, 20, 40, 60, 80



Creation of BST : Node class

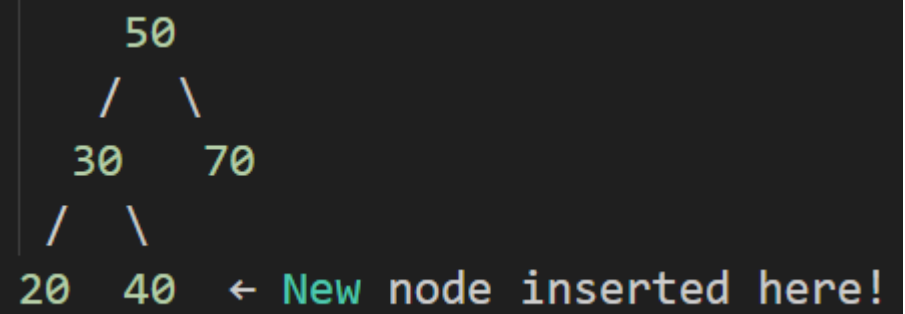
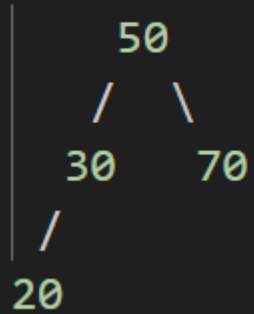


```
class Node {  
    int data;  
    Node left;  
    Node right;  
  
    // Constructor to create a new node  
    public Node(int data) {  
        this.data = data;  
        this.left = null;  
        this.right = null;  
    }  
}
```

Notice how smaller values always go left, larger values always go right!

Example of how iterative insertion works

Inserting 40 into the following Binary Search Tree (BST):



Step-by-step:

1. Create newNode with value 40
2. Start at root (50)
3. $40 < 50 \rightarrow$ Move to left child (30)
4. $40 > 30 \rightarrow$ Move to right child (null)
5. Found empty spot! Insert 40 as right child of 30

BST Creation : Time & Space Complexity

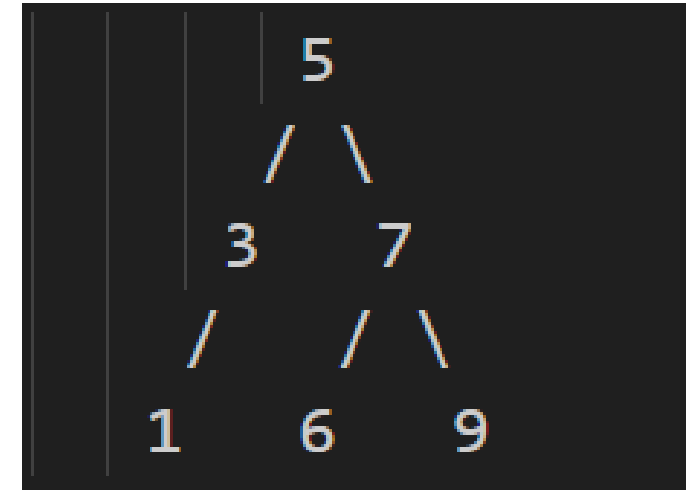
- **Time Complexity:**
 - **Best Case:** $O(\log n)$ - when the tree is balanced
 - **Average Case:** $O(\log n)$
 - **Worst Case:** $O(n)$ - when the tree becomes a skewed line (e.g., inserting sorted data)
- **Space Complexity:**
 - **$O(1)$** - Iterative approach uses constant extra space (just a few pointers)
 - The tree itself uses $O(n)$ space for n nodes

BST : Inorder Traversal

- Inorder traversal visits nodes in this order:

Left $\rightarrow$ Root $\rightarrow$ Right

- For a BST, this gives us nodes in ****sorted order****!
- Inorder will visit the nodes in this order: 1 3 5 6 7 9



BST : Inorder traversal (*Recursion)

```
public void inorderTraversal(Node root) {  
    if (root == null) {  
        return;  
    }  
  
    inorderTraversal(root.left); // Visit left subtree  
    System.out.print(root.data + " "); // Process current node  
    inorderTraversal(root.right); // Visit right subtree  
}
```

Inorder using Recursion

How does it work ?

- Each function call executes ALL its lines
- Recursive calls PAUSE the current function
- Functions RESUME from where they paused
- The recursion naturally goes deep left first
- The call stack remembers where to return AND what to do next

Refer to [inorder-traversal-notes](#) for detailed step by step explanation

Inorder Recursion

Time & Space Complexity

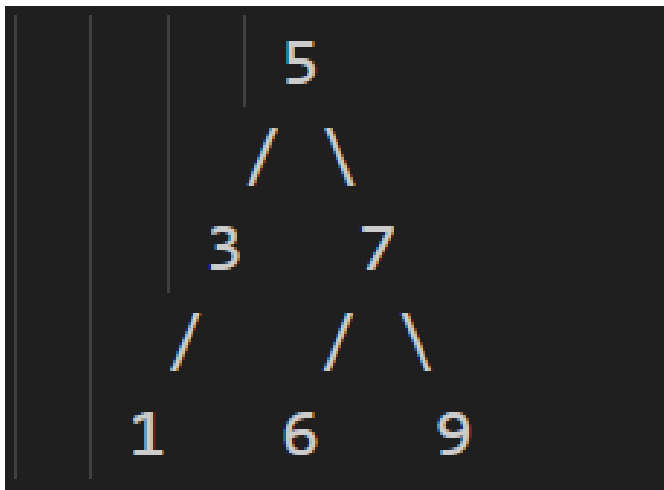
- **Time Complexity:** $O(n)$ - We visit each node exactly once
- **Space Complexity:** $O(h)$ - Maximum call stack depth is the height of the tree
 - Best case (balanced): $O(\log n)$
 - Worst case (skewed): $O(n)$

BST : Preorder Traversal

- Preorder traversal visits nodes in this order:

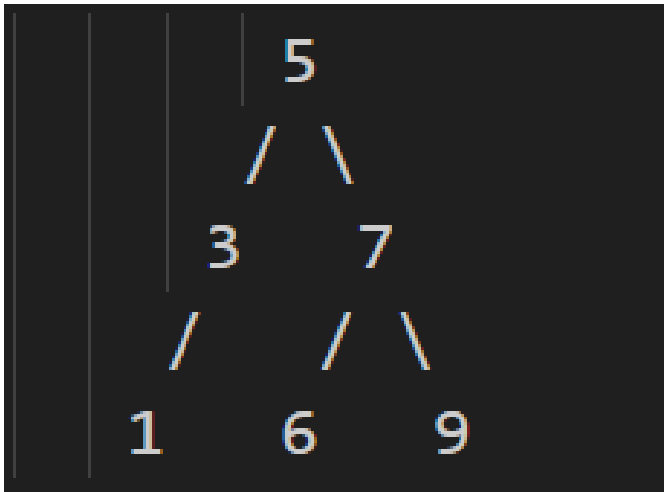
Root → Left → Right

- The key difference: we process the node first, followed by left subtree & then right subtree
- Preorder will visit the nodes in this order : 5 3 1 7 6 9



BST : Postorder Traversal

- Postorder traversal visits nodes in this order:
Left → Right → Root
- The key difference: we process the node LAST, after both subtrees
- Postorder will visit the nodes in this order : 1 3 6 9 7 5



BST : Postorder traversal (*Recursion)

```
public void postorderTraversal(Node root) {  
    if (root == null) {  
        return;  
    }  
  
    postorderTraversal(root.left); // Visit left subtree  
    postorderTraversal(root.right); // Visit right subtree  
    System.out.print(root.data + " "); // Process current node LAST  
}
```

Postorder using Recursion

How does it work ?

- The root prints LAST
- Each function waits through TWO recursive calls
- The "pause and resume" happens TWICE per node
- Children are always processed before parents
- The call stack manages the "waiting"

Refer to [postorder-traversal-notes](#) for detailed step by step explanation

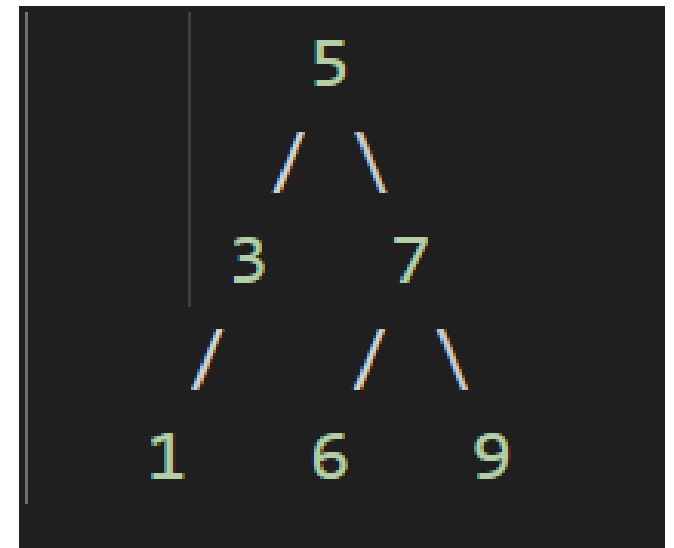
Postorder Recursion

Time & Space Complexity

- **Time Complexity:** $O(n)$ - We visit each node exactly once
- **Space Complexity:** $O(h)$ - Maximum call stack depth is the height of the tree
 - Best case (balanced): $O(\log n)$
 - Worst case (skewed): $O(n)$

Height of a Binary Tree

- The **height of a tree** is the longest path from the root to any leaf node (counting edges).
- This is a classic example of **bottom-up recursion** - we calculate heights from leaves and bubble up to the root!
- The height of the tree is 2



Code : Height of a Binary Tree

```
public int height(Node root) {  
    if (root == null) {  
        return -1; // Base case: null has height -1  
    }  
  
    int leftHeight = height(root.left); // Get left subtree height  
    int rightHeight = height(root.right); // Get right subtree height  
  
    return 1 + Math.max(leftHeight, rightHeight); // Current height  
}
```

The key insight: **Height = 1 + max(left height, right height)**

Why the formula works ?

```
return 1 + Math.max(leftHeight, rightHeight);
```

↑ ↑
| |
└─ Which path is longer?
└─ Add current edge/node

- If left is taller, we take that path and add 1 for current node
- If right is taller, we take that path and add 1 for current node
- If both equal (or both null), we take either and add 1

Height of a Binary Tree using Recursion

- **This is postorder recursion** - We process children FIRST, then use their results to calculate parent's value
- **Base case is critical** - $\text{height}(\text{null}) = -1$ makes the math work for leaf nodes ($1 + \max(-1, -1) = 0$)
- **The + 1 counts the current edge** - We add 1 for the edge from current node to its taller subtree
- **The max picks the taller path** - We want the LONGEST path, so we take the maximum height
- **Each node waits for BOTH children** - Just like postorder traversal, we need both recursive calls to complete

Height of Binary Tree : Time & Space Complexity

- **Time Complexity:** $O(n)$ - We visit each node exactly once
- **Space Complexity:** $O(h)$ – Call stack depth equals tree height
 - Best case (balanced): $O(\log n)$
 - Worst case (skewed): $O(n)$

Introduction to Hashing & Hash Tables

What is Hashing ?

Dewey Decimal System



- Think of a **library** that stores books using the :
Dewey Decimal System → Book title → number → shelf.
- Hashing does the same: key → index → array slot.

So, what is Hashing ?

Hashing is a technique to **convert** a given **key** into an **index** in a fixed-size array (**hash table**) using a **hash function**.

Hashing = $\text{hash}(\text{key}) \rightarrow \text{index} \rightarrow \text{array slot}.$

Why Hashing ?

- Fast access to data
- Search, Insert, Delete in **$O(1)$** average time
- Compared to:
 - Arrays: $O(n)$
 - Linked Lists: $O(n)$
 - Binary Search Trees: $O(\log n)$

```
HashMap<String, String> phoneBook = new HashMap<>();  
phoneBook.put("GoGo", "12345"); // internally hashed
```

Hashing – The Big Picture

- Key $\rightarrow$ Hash Function $\rightarrow$ Hash Table

The hash function converts the key into an index, and that index directly tells us where in the array to store the value.



Hashing : Another example

Step 1: Key

"GaGa"

Step 2: Apply Hash Function

$h(\text{"GaGa"}) = 478 \% 10 = 8$

Step 3: Insert into Hash Table

Index:	0	1	2	3	4	5	6	7	8	9
	+---	+---	+---	+---	+---	+---	+---	+---	+---	+---
Table:									GaGa	
	+---	+---	+---	+---	+---	+---	+---	+---	+---	+---

Hash Function

- A hash function converts any key into an array index (integer).
- Requirements of a Good Hash Function:
 - 1. **Deterministic:** Same input $\rightarrow$ Same output (always!)
 - 2. **Fast:** $O(1)$ computation
 - 3. **Uniform Distribution:** Spreads keys evenly across table
 - 4. **Minimize Collisions:** Different keys should rarely produce same index

Common Hash Function Types

1. Division Method

$\text{hash}(\text{key}) = \text{key} \% \text{tableSize}$

```
int hash(int key, int tableSize) {  
    return key % tableSize;  
}  
  
// Store employee ID 12345 in table of size 100  
hash(12345, 100) = 45 // Goes to index 45
```

Common Hash Function Types

2. Multiplication Method

$$\text{hash}(\text{key}) = (\text{key} * A) \% 1 * \text{tableSize}$$

```
int hash(int key, int tableSize) {  
    double A = 0.618034;  
    double temp = key * A;  
    temp = temp - Math.floor(temp); // Get fractional part  
    return (int)(tableSize * temp);  
}
```

Note : $A = \text{constant between } 0 \text{ and } 1$

// $A \approx 0.618034$ (golden ratio)

$5.5 \% 1 = 0.5$.

$5 - \text{Math.floor}(5.5)$ becomes $5.5 - 5 = 0.5$

Common Hash Function Types

3. An example of String hashing (Text keys)

```
int hashString(String key, int tableSize) {  
    int hash = 0;  
    for (int i = 0; i < key.length(); i++) {  
        hash = (31 * hash + key.charAt(i)) % tableSize;  
    }  
    return Math.abs(hash); // Handle negative overflow  
}  
  
// Example:  
hashString("alice", 10) → 7
```

Common Hash Function Types

4. Folding - Break key into parts, sum parts

Key = 12345678

Step 1: Split the key into equal parts

| 12 | 34 | 56 | 78

Step 2: Add the parts

| 12 + 34 + 56 + 78 = 180

Step 3: Map to table size

| Hash Index = 180 % 100 = 80

Common Hash Function Types

5. Mid Square - Square the key and extract the middle digits as the hash value

```
Key = 42
```

```
Step 1: Square the key
```

```
-----  
42 x 42 = 1764
```

```
Step 2: Extract middle digits
```

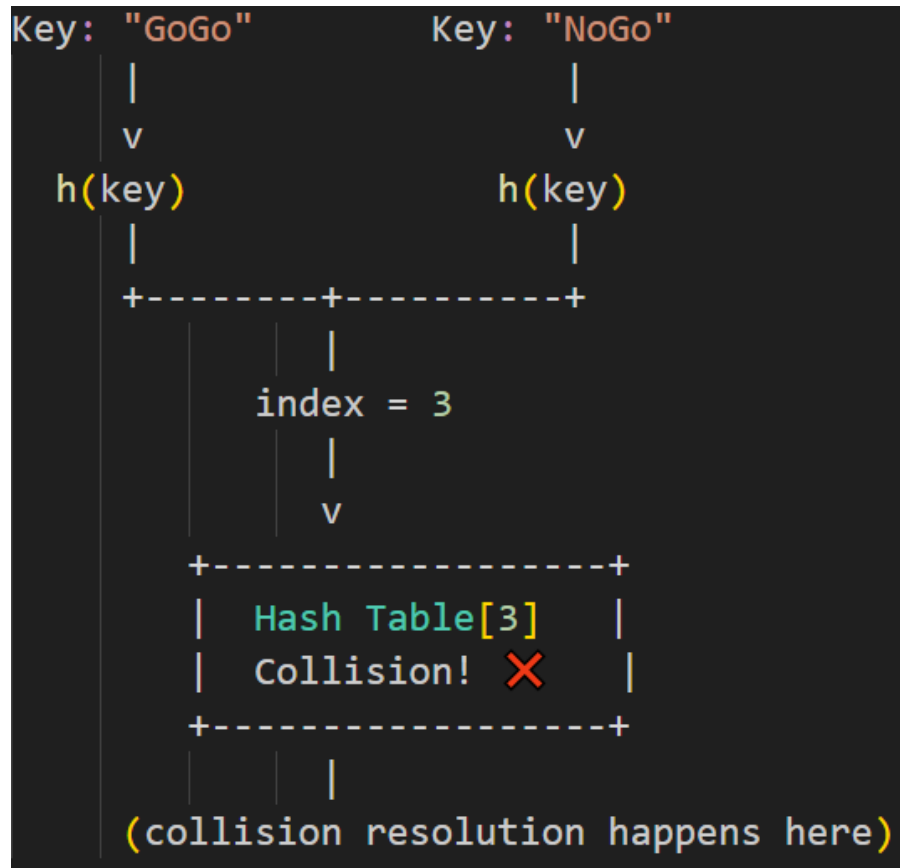
```
-----  
1 [ 7 6 ] 4 → Middle = 76
```

```
Step 3: Map to table size
```

```
-----  
Hash Index = 76 % 100 = 76
```

Hashing can lead to collisions

- Collision = When two different keys hash to the same index



Collision – Another example

Let's say we are hashing Person objects with :

- “Fred” $\rightarrow$ F=6, R=13, E=5, D=4, $6+13+5+4 =$ array index 28
- “Ned” $\rightarrow$ N=19, E=5, D=4 is $19+5+4=\text{index } 28$

If we try to put an item into a spot in the hash table that's occupied - collision

Collisions are unavoidable

Collision resolution techniques

- Open Addressing
 - All elements are stored directly inside the hash table array
 - Linear probing, quadratic probing, double hashing
- Closed Addressing (Separate Chaining)
 - Each array index stores a linked list of elements that hash to that index

Collision resolution

Separate Chaining(Closed addressing)

- Collisions are handled by growing a linked list at that index

Hash Function:

$h(\text{key}) = \text{key} \% 5$

Hash Table (size = 5)

Index

```
0 → null
1 → [21] → [16] → null
2 → [12] → null
3 → null
4 → [9] → [14] → null
```

*HashMap uses separate chaining
(Converts linked list → **balanced tree** if chain grows large)

Keys inserted:

$21 \% 5 = 1$

$16 \% 5 = 1$ ← collision handled by chaining

$12 \% 5 = 2$

$9 \% 5 = 4$

$14 \% 5 = 4$ ← collision handled by chaining

Collision resolution : Open addressing

- If the spot is taken, find another **empty spot** in the same table.
- Think of it like musical chairs - if your chair is taken, you look for the next available one!
- So there are 3 techniques under open addressing

Open Addressing : Linear Probing

- Simple Rule
 - If index is full $\rightarrow$ try index+1
 - If index+1 is full $\rightarrow$ try index+2
 - If index+2 is full $\rightarrow$ try index+3
 - Keep going until you find an empty spot!

```
Seats: [_] [_] [_] [_] [_] [X] [_] [_] [_] [_]
      0  1  2  3  4  5  6  7  8  9
           ↑
        Seat 5 taken!

Try seat 6  $\rightarrow$  Empty! Sit there ✓
```

Open Addressing : Linear Probing

- Example : Table size = 10, Hash function: $\text{key} \% 10$

Empty Table:

Index:	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	[_]	[_]	[_]	[_]	[_]	[_]	[_]	[_]	[_]	[_]

*Step 1: Insert 25 , Hash: $25 \% 10 = 5$, Index 5 is empty

Index:	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	[_]	[_]	[_]	[_]	[_]	[25]	[_]	[_]	[_]	[_]

*Step 2: Insert 35 , Hash: $35 \% 10 = 5$, Index 5 is full! COLLISION!

Try $5+1 = 6 \rightarrow$ Empty! Place it there!

Index:	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	[_]	[_]	[_]	[_]	[_]	[25]	[35]	[_]	[_]	[_]

						↑	↑			
						wanted	went here			

Open Addressing : Linear Probing

- Example : Table size = 10, Hash function: $\text{key} \% 10$

Step 3: Insert 45 : Hash: $45 \% 10 = 5$

Index 5 is full! Try 6 $\rightarrow$ also full!

Try $6+1 = 7 \rightarrow$ Empty!

Index:	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
						[25]	[35]	[45]		
						↑		↑		
						wanted		went here		

Step 4: Insert 55 ,Hash: $55 \% 10 = 5$

Index 5 full $\rightarrow$ try 6 full $\rightarrow$ try 7 full $\rightarrow$ try 8 $\rightarrow$ empty

Index:	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
						[25]	[35]	[45]	[55]	

Search for 45:

Hash: $45 \% 10 = 5$

Check index 5 $\rightarrow$ Found 25 (not it!)

Check index 6 $\rightarrow$ Found 35 (not it!)

Check index 7 $\rightarrow$ Found 45 $\checkmark$ FOUND!

erved.

Linear Probing : Disadvantages

- When many collisions happen, they create "clusters" (chains of filled slots)

```
Insert: 15, 25, 35, 45, 55 (all hash to 5)
Result: [_] [_] [_] [_] [_] [15] [25] [35] [45] [55]
        |____ Cluster! _____|
```

- More collisions = more searches = slower operations
- Like a traffic jam - one accident causes more delays!

Quadratic Probing

Jump Further Each Time

- **The Main Idea -**

If index is full $\rightarrow$ try index $+ 1^2$

Still full? $\rightarrow$ try index $+ 2^2$

Still full? $\rightarrow$ try index $+ 3^2$

Jump by squares: 1, 4, 9, 16, 25...

- **Advantage -** Spreads out better! Avoids creating long clusters

Quadratic Probing

Jump Further Each Time

- If parking spot #5 is taken
 - Instead of checking 6, 7, 8...
 - We jump: $5 \rightarrow 6 (5+1^2) \rightarrow 9 (5+2^2) \rightarrow 14 (5+3^2) \rightarrow \dots$
- **We are exploring the parking lot more efficiently!**

Linear Probing vs. Quadratic Probing

- 25, 35, 45, 55 (Table size = 10, key % 10)

Linear Probing:

25 goes to index 5
35 tries 5 → goes to 6
45 tries 5, 6 → goes to 7
55 tries 5, 6, 7 → goes to 8

Result: [] [] [] [] [] [25] [35] [45] [55] []
 └── Cluster ─┘

Quadratic Probing:

25 → hash(25) = 5 → index 5

35 → hash(35) = 5 (collision!)
Try: $5 + 1^2 = 6$ (empty)

45 → hash(45) = 5 (collision!)
Try: $5 + 1^2 = 6$ (full!)
Try: $5 + 2^2 = 5 + 4 = 9$ (empty)

55 → hash(55) = 5 (collision!)
Try: $5 + 1^2 = 6$ (full!)
Try: $5 + 2^2 = 9$ (full!)
Try: $5 + 3^2 = 5 + 9 = 14 \% 10 = 4$ (empty)

Result: [] [] [] [] [55] [25] [35] [] [] [45]

Much more spread out! No cluster!

Double Hashing : Two different Recipes

- **The Main Idea:**
 - Use **TWO** hash functions:
 - hash1: Finds starting position
 - hash2: Decides how far to jump each time
- hash1 = "Which street do you start on?"
- hash2 = "How many blocks do you walk each time?"

```
Table size = 10
```

```
hash1(key) = key % 10           // Starting position
```

```
hash2(key) = 1 + (key % 7)      // Jump size (must not be 0!)
```

Double Hashing : Example

$\text{hash1}(\text{key}) = \text{key} \% 10$, $\text{hash2}(\text{key}) = 1 + (\text{key} \% 7)$

Insert 25:

$\text{hash1}(25) = 25 \% 10 = 5$ // Start at 5
 $\text{hash2}(25) = 1 + (25 \% 7) = 5$ // Jump by 5 each time

Try: 5 (empty)

Table: [] [] [] [] [] [25] [] [] [] []

Insert 35:

$\text{hash1}(35) = 35 \% 10 = 5$ // Start at 5
 $\text{hash2}(35) = 1 + (35 \% 7) = 2$ // Jump by 2 each time

Try: 5 (full!)

Try: $5 + 2 = 7$ (empty)

Table: [] [] [] [] [] [25] [] [35] [] []

reserved.

Double Hashing : Example contd...

$\text{hash1}(\text{key}) = \text{key} \% 10$, $\text{hash2}(\text{key}) = 1 + (\text{key} \% 7)$

```
Insert 45:
hash1(45) = 45 % 10 = 5      // Start at 5
hash2(45) = 1 + (45 % 7) = 6  // Jump by 6 each time

Try: 5 (full!)
Try: 5 + 6 = 11 % 10 = 1 (empty)

Table: [_] [45] [_] [_] [_] [25] [_] [35] [_] [_]

Insert 55:
hash1(55) = 55 % 10 = 5      // Start at 5
hash2(55) = 1 + (55 % 7) = 4  // Jump by 4 each time

Try: 5 (full!)
Try: 5 + 4 = 9 (empty)

Table: [_] [45] [_] [_] [_] [25] [_] [35] [_] [55]
```

- Each key gets its own unique "jump pattern"!
- Different patterns = Less chance of collision!

Linear vs. Quadratic vs. Double

Insert 5 numbers that all hash to index 5

LINEAR PROBING:

[_][_][_][_][_][A][B][C][D][E] // Cluster at 5-9



The diagram shows a horizontal array of 10 slots. The first five slots are empty, represented by underscores. The next five slots contain the letters A, B, C, D, and E. A bracket is drawn underneath the slots containing A, B, C, D, and E, indicating a cluster of five elements starting at index 5.

QUADRATIC PROBING:

[E][_][_][_][D][A][B][_][_][C] // More spread out

DOUBLE HASHING:

[C][_][E][_][B][A][_][D][_][_] // Best distribution!

Hash tables usage in the Real World

- **Caching:** Redis, Memcached
- **Databases:** Indexing, JOIN operations
- **Security:** Password storage
- **Compilers:** Symbol tables
- **DNS:** Domain $\rightarrow$ IP mapping
- **E-commerce:** Shopping cart sessions

Key Takeaways

- Hash tables give **$O(1)$** average time
- **Collisions are inevitable**
- Double hashing > Quadratic > Linear

A peek into HashMap

HashMap contains:

1. An array (the actual hash table)
2. Nodes that form linked lists (for collision handling via chaining)
3. A hash function

```
// Simplified version of what's inside HashMap
class HashMap<K, V> {
    // An array of "buckets" (this is the hash table!)
    Node<K,V>[] table;

    // Each bucket contains a linked list (or tree)
    static class Node<K,V> {
        final int hash;
        final K key;
        V value;
        Node<K,V> next; // For chaining
    }

    // Hash function
    int hash(Object key) {
        return (key == null) ? 0 : key.hashCode() ^ (key.hashCode() >>> 16);
    }
}
```

Key Takeaways : HashMap

- HashMap has an internal array - This is the hash table.
- `put(key, value)` → Calculates hash → Finds index → Stores in array
- `get(key)` → Calculates hash → Finds index → Retrieves from array
- Both operations are $O(1)$ because array access is $O(1)$
- Collisions are handled with chaining (linked lists at each index)